Laporan Tugas Besar 1

Sistem Persamaan Linier, Determinan, dan Penerapannya

IF2123 Aljabar Linier dan Geometri



Disusun oleh:

Ghina Emelia Yantes	13525119
Sahla Nailah Salsabilla	13525134
Nayla Putri Ghaisani	13525140

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

1 Pendahuluan **2**

1.1 Deskripsi Masalah 2

1.2 Latar Belakang 2

1.3 Tujuan 3

2 Dasar Teori **4**

2.1 Sistem Persamaan Linear 4

2.1.1 Metode Eliminasi Gauss 4

2.1.2 Metode Eliminasi Gauss-Jordan 4

2.1.3 Metode Matriks Balikan 5

2.1.4 Kaidah Cramer 5

2.2 Determinan Matriks 5

2.2.1 Determinan Kofaktor 6

2.2.2 Determinan Reduksi Baris (OBE) 6

2.3 Invers Matriks 7

2.3.1 Inverse Augmented 7

2.3.2 Inverse Adjoint 7

2.4 Interpolasi Polinomial 7

2.5 Natural Cubic Spline Interpolation 8

2.5.1 Tridiagonal Matrix System for Cubic Splines 9

2.6 Splines Regression 10

3 Implementasi **13**

3.1 Struktur Kelas 13

3.1.1 Paket `algeo.matrix` 13

3.1.2 Paket `algeo.spl` 14

3.1.3 Paket `algeo.determinant` 15

3.1.4 Paket `algeo.inverse` 15

3.1.5 Paket `algeo.common` 16

3.1.6 Paket `algeo.interpolation` dan `algeo.spline` 16

3.1.7 Paket `algeo.io` dan kelas `App` 17

3.2 Arsitektur Pustaka 18
3.2.1 Pembagian Paket 18
3.2.2 Prinsip Perancangan 19
3.2.3 Struktur Direktori Proyek 19
3.3 Alur Program 20
3.3.1 Alur Utama 20
3.3.2 Pemilihan Sumber Data 21
3.3.3 Alur Sistem Persamaan Linear 21
3.3.4 Alur Determinan dan Invers 22
3.3.5 Alur Interpolasi dan Regresi 22
3.4 Build Tool dan Packaging 23
3.4.1 Maven sebagai Build Tool 23
3.4.2 Perintah Build dan Menjalankan Program 23
3.4.3 Pembuatan Berkas .jar yang Dapat Dieksekusi 24

4 Eksperimen **25**
4.1 Kasus Uji 4.1: Determinan Matriks 25
4.1.1 Kasus Uji 4.1.1: Matriks A Berukuran 4×4 25
4.1.2 Kasus Uji 4.1.2: Matriks A Berukuran 5×5 26
4.1.3 Kasus Uji 4.1.3: Matriks A Berukuran 4×4 27
4.2 Kasus Uji 4.2: Invers Matriks 28
4.2.1 Kasus Uji 4.2.1: Matriks A Berukuran 4×4 28
4.2.2 Kasus Uji 4.2.2: Matriks A Berukuran 5×5 29
4.3 Kasus Uji 4.3: Sistem Persamaan Linier ($Ax = b$) 30
4.3.1 Kasus Uji 4.3.1: Matriks A Berukuran 4×4 30
4.3.2 Kasus Uji 4.3.2: Matriks A Berukuran 3×5 32
4.3.3 Kasus Uji 4.3.3: Matriks A Berukuran 4×4 34
4.3.4 Kasus Uji 4.3.4: Matriks *discrete Laplacian* satu dimensi 35
4.4 Kasus Uji 4.4: Sistem Persamaan Linier Berbentuk Matriks Augmented . . 39
4.4.1 Kasus Uji 4.4.1: Matriks augmented A Berukuran 4×4 39
4.4.2 Kasus Uji 4.4.2: Matriks augmented A Berukuran 4×6 41
4.5 Kasus Uji 4.5: Sistem Persamaan Linier Berbentuk Umum 42
4.5.1 Kasus Uji 4.5.1: SPL 4×4 42
4.5.2 Kasus Uji 4.5.2: SPL 9×9 44
4.6 Kasus Uji 4.6: Aplikasi Sistem Persamaan Linier 46

4.7 Kasus Uji 4.7: Interpolasi Polinomial 48
4.7.1 Kasus Uji 4.7.1: Tujuh Titik Data (Domain $[0{,}2;\ 1{,}4]$) 48
4.7.2 Kasus Uji 4.7.2: Enam Titik Data (Domain $[0;\ 10]$) 49
4.7.3 Kasus Uji 4.7.3: Tiga Titik Data 50
4.8 Kasus Uji 4.8: Studi Interpolasi Natural Cubic Spline 51
4.8.1 Kasus Uji 4.8.1: Lima Titik Data 51
4.9 Kasus Uji 4.9: Studi Kasus Regresi Spline 52
4.9.1 Kasus Uji 4.9.1 52

5 Penutup **55**
5.1 Kesimpulan 55
5.2 Saran 55
5.3 Refleksi Anggota Kelompok 55

6 Daftar Pustaka **57**

7 Lampiran **58**
7.1 Repositori GitHub 58
7.2 Bonus LaTeX 58
7.3 Pernyataan Integritas Akademik 58

1 Pendahuluan

1.1 Deskripsi Masalah

Di pelosok Snezhnaya, seorang peneliti muda bernama Mitya tengah berusaha keras memimpin eksperimen krusial berskala raksasa: Proyek Stuzha. Proyek ini digadang-gadang mampu mengendalikan fluktuasi energi ekstrem di daratan Teyvat. Sayangnya, impian besar tersebut terbentur tembok tebal: Mitya masih mengandalkan kalkulator saku tua yang sering error tiap kali dipaksa menyelesaikan kalkulasi berdimensi tinggi.



Gambar 1.1: Berkenalan dengan Mitya
Sumber: Arsip Penulis

Setelah menjelajahi perpustakaan Sumeru Akademiya hingga menyisir kabar burung ke seantero Teyvat, Mitya mendengar reputasi kalian sebagai praktisi komputasi dan aljabar linier paling piawai. Ia pun mengirimkan pesan ke Adventures Guild dan memberikan komisi perangkat lunak sekelas Photomath versi Teyvat yaitu sebuah mesin kalkulasi otomatis yang sanggup melumat sistem matriks dan persamaan linier rumit demi kelangsungan Proyek Stuzha.

Aljabar linier bukan sekadar teori di atas kertas papirus; ini adalah fondasi komputasi modern yang menggerakkan simulasi rekayasa, pemodelan aliran energi, analisis data, hingga dinamika sistem kompleks. Agar kalkulasi Proyek Stuzha tidak runtuh di tengah jalan, kalian ditugaskan merancang dan membangun pustaka (*library*) komputasi aljabar linier mandiri berbasis bahasa pemrograman **Java** yang modular, tangguh, dan terdokumentasi rapi.

1.2 Latar Belakang

Perkembangan teknologi rekayasa modern dan simulasi skala besar menuntut landasan komputasi yang tidak hanya akurat secara teoretis, tetapi juga tangguh dalam eksekusi praktis. Dalam perancangan sistem fisik yang kompleks, seperti pengoperasian infrastruktur pengendali energi ekstrem. Analisis matematis tidak lagi dapat diselesaikan secara manual atau menggunakan perangkat komputasi yang terbatas. Kompleksitas variabel yang saling terikat dalam jumlah besar menciptakan matriks dimensi tinggi yang rentan terhadap akumulasi

galat jika ditangani dengan instrumen konvensional.

Kendala fundamental ini dihadapi dalam operasional Proyek Stuzha, di mana ketergantungan pada alat hitung konvensional yang usang memicu hambatan serius dalam validasi data dan pengujian kestabilan sistem. Kegagalan dalam menyelesaikan Sistem Persamaan Linier (SPL) berdimensi besar, perhitungan determinan, serta invers matriks secara presisi dapat berakibat fatal pada kalkulasi keseimbangan sirkuit daya. Oleh karena itu, diperlukan transisi menuju sistem komputasi terotomatisasi yang terstruktur.

Sebagai respons atas kebutuhan mendesak tersebut, pengembangan sebuah *library* komputasi aljabar linier mandiri berbasis **Java** menjadi solusi arsitektural yang krusial. Pendekatan berbasis objek (*object-oriented programming*) dipilih untuk memastikan modularitas kode, skalabilitas, dan kemudahan pemeliharaan. Pustaka ini dirancang untuk mengintegrasikan tiga pilar utama kebutuhan komputasi proyek:

1. Penyelesaian Aljabar Matriks dan SPL: Sebagai kerangka dasar untuk menganalisis kestabilan instrumen dan distribusi daya secara simultan.
2. Metode Interpolasi dan Regresi Numerik: Memungkinkan pemetaan jalur transfer energi serta pemodelan kontur data eksperimental yang bersifat non-linear dengan tingkat galat minimal.

Melalui perancangan pustaka ini, implementasi teori aljabar linier ditransformasikan menjadi perangkat lunak yang andal, siap menghadapi skenario uji ekstrem, dan menjadi fondasi komputasi yang berkelanjutan bagi instrumen riset masa depan.

1.3 Tujuan

1. Mengimplementasikan metode penyelesaian SPL, determinan, invers matriks, interpolasi polinomial, Natural Cubic Spline Interpolation, dan regresi Spline secara mandiri menggunakan Java.
2. Membangun pustaka (library) aljabar linier yang terstruktur secara modular, bersih, dan terdokumentasi dengan baik. Tidak menggunakan library perhitungan matriks, regresi, atau interpolasi instan.
3. Menguji keandalan pustaka pada berbagai kasus uji analitis dan restorasi citra, serta menyusun laporan analisis hasil perhitungannya.

2 Dasar Teori

2.1 Sistem Persamaan Linear

Sistem Persamaan Linear (SPL) adalah himpunan berhingga dari persamaan-persamaan linear dengan variabel-variabel yang sama. Bentuk umum dari SPL dengan m persamaan dan n variabel adalah

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2 \\ \qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad \cdots \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n \end{cases}$$

atau, dalam bentuk perkalian matriks (kiri) dan matriks augmented (kanan)

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}}_{b} \quad \rightarrow \quad \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} & b_n \end{bmatrix}$$

Solusi dari SPL dapat berupa solusi tunggal, banyak solusi, atau tidak ada solusi. Metode penyelesaian yang umum digunakan antara lain eliminasi Gauss, eliminasi Gauss-Jordan, kaidah Cramer, dan metode matriks balikan.

2.1.1 Metode Eliminasi Gauss

Eliminasi Gauss adalah metode penyelesaian sistem persamaan linear dengan mengubah matriks augmented ke bentuk eselon baris, lalu mencari nilai variabel menggunakan *substitusi mundur*. Langkah-langkah utamanya meliputi:

1. Menyusun sistem persamaan linear ke dalam bentuk matriks augmented $[A \mid b]$.
2. Melakukan Operasi Baris Elementer (OBE) melalui fase *eliminasi maju* (*forward elimination*) hingga elemen di bawah diagonal utama bernilai nol (membentuk matriks segitiga atas).
3. Menerapkan proses *substitusi mundur* (*back-substitution*) dari baris paling bawah ke atas untuk mendapatkan nilai setiap variabel $(x_1, x_2, \ldots, x_n)$.

2.1.2 Metode Eliminasi Gauss-Jordan

Eliminasi Gauss-Jordan merupakan pengembangan lanjutan dari eliminasi Gauss. Metode ini menambahkan fase eliminasi mundur di dalam operasi baris sehingga matriks berubah menjadi bentuk eselon baris tereduksi (matriks identitas). Langkah-langkahnya meliputi:

1. Menyusun sistem persamaan linear ke dalam bentuk matriks augmented $[A \mid b]$.
2. Melakukan eliminasi maju untuk membentuk segitiga atas.
3. Melanjutkan eliminasi mundur (masih menggunakan OBE) untuk membuat elemen di atas diagonal utama bernilai nol dan diagonal utamanya bernilai satu.
4. Nilai variabel atau solusi akhir dapat langsung dibaca dari kolom terakhir matriks tanpa perlu melalui proses substitusi mundur manual.

2.1.3 Metode Matriks Balikan

Metode matriks balikan (invers) digunakan untuk menyelesaikan sistem persamaan linear yang dapat direpresentasikan dalam bentuk persamaan matriks $AX = B$. Langkah-langkah utamanya meliputi:

1. Mengubah sistem persamaan linear ke dalam bentuk matriks $AX = B$, di mana A adalah matriks koefisien, X adalah matriks variabel, dan B adalah matriks konstanta.
2. Memastikan bahwa matriks A adalah matriks persegi dan memiliki balikan (invers), yaitu determinan $A \neq 0$.
3. Menghitung invers dari matriks A (A^{-1}).
4. Mendapatkan nilai variabel dengan mengalikan invers matriks A dengan matriks konstanta B melalui rumus $X = A^{-1}B$.

2.1.4 Kaidah Cramer

Kaidah Cramer adalah metode penyelesaian sistem persamaan linear menggunakan konsep determinan matriks. Metode ini efektif untuk sistem dengan ukuran kecil (2×2 atau 3×3). Langkah-langkah utamanya meliputi:

1. Menyusun sistem persamaan linear ke dalam bentuk matriks $AX = B$.
2. Menghitung determinan dari matriks koefisien A, yaitu $\det(A)$, dengan syarat $\det(A) \neq 0$.
3. Membentuk matriks A_i dengan cara mengganti kolom ke-i pada matriks A dengan matriks konstanta B, lalu menghitung determinannya ($\det(A_i)$) untuk setiap variabel.
4. Menentukan nilai setiap variabel (x_i) menggunakan rumus:

$$x_i = \frac{\det(A_i)}{\det(A)}$$

2.2 Determinan Matriks

Determinan adalah sebuah nilai skalar khusus yang dapat dihitung dari elemen-elemen sebuah matriks persegi. Determinan dari matriks A dinotasikan sebagai $\det(A)$ atau $|A|$. Nilai

determinan sangat krusial karena dapat mengindikasikan apakah matriks tersebut memiliki invers. Beberapa metode untuk menghitung determinan matriks mencakup metode ekspansi kofaktor dan metode reduksi baris (OBE).

2.2.1 Determinan Kofaktor

Determinan kofaktor (atau ekspansi kofaktor) adalah metode menghitung determinan matriks persegi dengan cara mengembangkannya sepanjang baris atau kolom tertentu. Langkah-langkah dan konsep utamanya meliputi:

1. Memilih baris atau kolom tertentu pada matriks A (biasanya dipilih yang memiliki banyak unsur nol untuk mempermudah perhitungan).
2. Menghitung minor (M_{ij}), yaitu determinan dari submatriks berukuran $(n-1)\times(n-1)$ yang diperoleh dengan menghapus baris ke-i dan kolom ke-j dari matriks A.
3. Menentukan kofaktor C_{ij} melalui rumus:

$$C_{ij} = (-1)^{i+j} M_{ij}$$

4. Menjumlahkan hasil kali setiap elemen pada baris atau kolom pilihan dengan kofaktorkannya. Sebagai contoh, ekspansi sepanjang baris pertama dirumuskan sebagai:

$$\det(A) = a_{11}C_{11} + a_{12}C_{12} + a_{13}C_{13} + \cdots + a_{1n}C_{1n}$$

2.2.2 Determinan Reduksi Baris (OBE)

Determinan reduksi baris adalah metode menghitung determinan dengan mereduksi matriks persegi menjadi bentuk matriks segitiga atas menggunakan Operasi Baris Elementer (OBE). Metode ini jauh lebih efisien untuk matriks berukuran besar. Aturan perubahan nilai determinan akibat OBE meliputi:

1. Pertukaran baris: Jika dua baris matriks ditukar posisinya, maka nilai determinan akan bertanda berlawanan (dikalikan dengan -1).
2. Perkalian baris dengan skalar: Jika seluruh elemen pada suatu baris dikalikan dengan skalar k, maka determinan matriks baru menjadi k kali determinan semula.
3. Penjumlahan kelipatan baris: Menambahkan kelipatan suatu baris ke baris lainnya tidak mengubah nilai determinan sama sekali.

Setelah matriks berhasil diubah menjadi bentuk segitiga atas (elemen di bawah diagonal utama bernilai nol), determinan dapat dihitung dengan mudah melalui hasil kali seluruh elemen pada diagonal utamanya:

$$\det(A) = a_{11}a_{22}a_{33}\ldots a_{nn}$$

2.3 Invers Matriks

Invers dari sebuah matriks persegi A adalah matriks A^{-1} yang memenuhi properti $AA^{-1} = A^{-1}A = I$, di mana I adalah matriks identitas. Sebuah matriks hanya memiliki invers jika dan hanya jika determinannya tidak nol ($\det(A) \neq 0$). Matriks yang memiliki invers disebut matriks *invertible* atau non-singular. Perhitungan invers dari suatu matriks dapat dilakukan dengan metode matriks adjoin maupun metode eliminasi Gauss-Jordan.

2.3.1 Inverse Augmented

Invers matriks menggunakan metode augmented (atau eliminasi Gauss-Jordan) dilakukan dengan mereduksi matriks asal berdampingan dengan matriks identitas. Langkah-langkahnya meliputi:

1. Menuliskan matriks augmented yang menggabungkan matriks A dan matriks identitas I dengan bentuk $[\ A \mid I\ ]$.
2. Melakukan Operasi Baris Elementer (OBE) pada seluruh baris matriks augmented tersebut.
3. Melanjutkan OBE hingga sisi kiri matriks yang awalnya berupa matriks A berubah sepenuhnya menjadi matriks identitas I.
4. Membaca hasil akhir pada sisi kanan matriks augmented, di mana bagian tersebut secara otomatis telah berubah menjadi matriks invers (A^{-1}), sehingga bentuk akhirnya menjadi $[\ I \mid A^{-1}\ ]$.

2.3.2 Inverse Adjoint

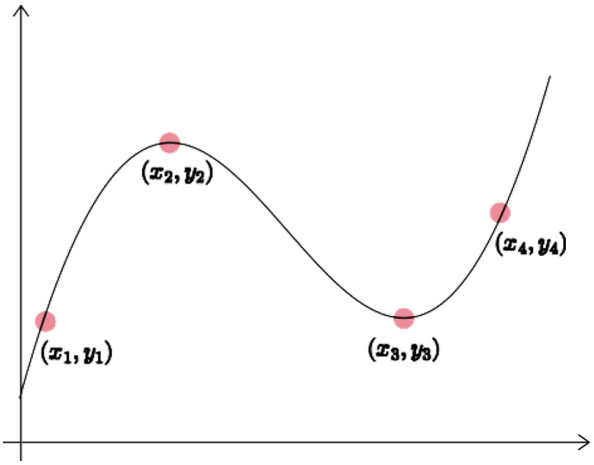
Invers matriks menggunakan adjoin dihitung dengan memanfaatkan nilai determinan dan matriks kofaktor dari suatu matriks persegi. Langkah-langkahnya meliputi:

1. Menghitung determinan matriks A, yaitu $\det(A)$, dan memastikan bahwa $\det(A) \neq 0$ agar matriks tersebut memiliki invers.
2. Menentukan matriks kofaktor dari A dengan mencari kofaktor untuk setiap elemen matriks tersebut.
3. Menentukan matriks adjoin, yaitu adj(A), dengan cara mentranspose-kan matriks kofaktor yang telah diperoleh.
4. Menghitung invers matriks A menggunakan rumus:

$$A^{-1} = \frac{1}{\det(A)}\text{adj}(A)$$

2.4 Interpolasi Polinomial

Interpolasi polinomial adalah metode untuk menentukan suatu polinomial $P_n(x)$ berderajat n yang melalui $n+1$ titik data $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$, x_i diurutkan dengan x_0 terkecil



Gambar 2.1: Gambar 2: Contoh interpolasi polinomial melalui empat titik data.

dan x_n terbesar. Secara umum, polinomial tersebut dapat dituliskan sebagai:

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

dengan koefisien $a_0, a_1, \ldots, a_n$ yang ditentukan sedemikian rupa sehingga $P_n(x_{\text{perkiraan}}) = y_{\text{perkiraan}}$ untuk semua nilai $x_0 < x_{\text{perkiraan}} < x_n$. Derajat persamaan polynom ditentukan oleh jumlah titik data yang diberikan dikurangi satu.

Koefisien polinomial dapat dicari dengan menyulihkan titik-titik data ke dalam persamaan polinomial dan menyelesaikan sistem persamaan linear yang dihasilkan dengan metode eliminasi Gauss yang dijelaskan sebelumnya. Kita dapat menyusun sistem persamaan linear sebagai berikut. o

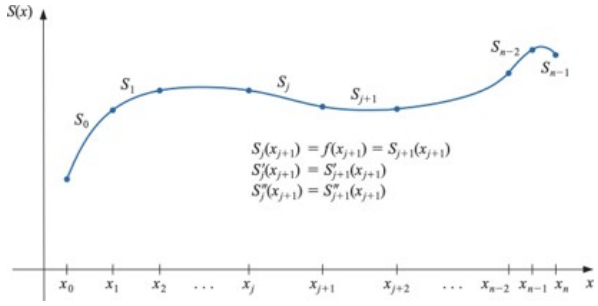
$$\begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0 \\ a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_n x_1^n = y_1 \\ \qquad\qquad\qquad\qquad\qquad\qquad\quad \vdots \\ a_0 + a_1 x_n + a_2 x_n^2 + \cdots + a_n x_n^n = y_n \end{cases}$$

Persamaan-persamaan di atas dapat dipetakan menjadi sebuah permasalahan sistem persamaan linear, sehingga dapat diselesaikan dengan menggunakan metode eliminasi Gauss untuk menyelesaikan

sistem tersebut dan menemukan koefisien polinomial $a_0, a_1, \ldots, a_n$. Dari polynom $P_n(x)$ yang dibentuk dengan koefisien tersebut, kita dapat memperkirakan nilai y untuk nilai x yang berada di antara titik-titik data yang diberikan (interpolasi) maupun di luar titik-titik data tersebut (ekstrapolasi).

2.5 Natural Cubic Spline Interpolation

Interpolasi Spline Kubik (*Cubic Spline Interpolation*) adalah metode interpolasi yang mengonstruksi kurva mulus melalui sekumpulan titik data diskret menggunakan potongan-potongan polinomial berderajat tiga (kubik) di setiap interval antar-titik. Berbeda dengan interpolasi polinomial tunggal orde tinggi yang rentan mengalami osilasi liar di ujung interval, *spline* menjaga kelengkungan kurva tetap stabil dan kontinu hingga turunan kedua.



Gambar 2.2: Gambar : Contoh Interpolasi Spline Kubik
Sumber: R. L. Burden & J. D. Faires, *Numerical Analysis*

Definisi dari *cubic spline interpolation* S untuk sebuah fungsi f yang terdefinisi di $[a, b]$ dengan sekumpulan titik data **terurut** $a = x_0 < x_1 < \cdots < x_n = b$ memenuhi kondisi berikut:

a. $S(x)$ merupakan potongan polinomial kubik $S_j(x)$ pada setiap subinterval $[x_j, x_{j+1}]$ untuk setiap $j = 0, 1, \ldots, n-1$ (terdapat n subinterval).

b. $S_j(x_j) = f(x_j)$ dan $S_j(x_{j+1}) = f(x_{j+1})$ untuk setiap $j = 0, 1, \ldots, n-1$.
(Kurva harus melalui nilai fungsi pada titik-titik ujung tiap subinterval).

c. $S_{j+1}(x_{j+1}) = S'j(x_{j+1})$ untuk setiap $j = 0, 1, \ldots, n-2$.
(Kurva bersambung secara kontinu tanpa putus di setiap simpul interior).

d. $S'_{j+1}(x_{j+1}) = S"j(x_{j+1})$ untuk setiap $j = 0, 1, \ldots, n-2$.
(Kemiringan kurva dari sisi kiri dan kanan bernilai sama di setiap simpul interior).

e. $S"_{j+1}(x_{j+1}) = S"j(x_{j+1})$ untuk setiap $j = 0, 1, \ldots, n-2$.
(Kelengkungan (curvature) kurva dari sisi kiri dan kanan bernilai sama di setiap simpul interior).

f. $S"_(x_0) = S"(x_n)$ untuk *natural spline*.
(Kurva tidak memiliki gaya lentur pada kedua ujung terluar sehingga berperilaku menyerupai garis lurus di luar rentang data).

Penjelasan matematis lebih mendalam mengenai penurunan rumus, pembentukan sistem persamaan tridiagonal, serta visualisasi grafik interpolasi spline kubik dapat dirujuk pada buku Numerical Analysis karya Burden dan Faires.

2.5.1 Tridiagonal Matrix System for Cubic Splines

Untuk menentukan kurva kubik yang mulus, turunan kedua pada setiap titik data ($z_i = S''(x_i)$) dihitung melalui suatu sistem persamaan linear khusus yang membentuk matriks tridiagonal. Sistem ini dirumuskan sebagai berikut:

1. Persamaan kontinuitas turunan pertama antar segmen polinomial menghasilkan hubungan antarnilai turunan kedua:

$$h_{i-1}z_{i-1} + 2(h_{i-1} + h_i)z_i + h_i z_{i+1} = 6\left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}}\right)$$

di mana $h_i = x_{i+1} - x_i$ adalah jarak antar titik data.

2. Untuk *natural cubic spline*, nilai turunan kedua pada ujung-ujung interval bernilai nol ($z_0 = 0$ dan $z_n = 0$).

3. Sistem persamaan linear di atas dapat direpresentasikan dalam bentuk matriks tridiagonal $Hz = b$:

$$\begin{pmatrix} 2(h_0+h_1) & h_1 & 0 & \dots & 0 \\ h_1 & 2(h_1+h_2) & h_2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & h_{n-2} & 2(h_{n-2}+h_{n-1}) \end{pmatrix} \begin{pmatrix} z_1 \\ z_2 \\ \vdots \\ z_{n-1} \end{pmatrix} = \begin{pmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \end{pmatrix}$$

4. Matriks koefisien di atas bersifat tridiagonal (hanya elemen pada diagonal utama, super-diagonal, dan sub-diagonal yang tidak bernilai nol), sehingga dapat diselesaikan secara sangat efisien menggunakan algoritma eliminasi Gauss khusus (seperti algoritma Thomas).

2.6 Splines Regression

Splines Regression adalah metode pemodelan non-parametrik yang memetakan hubungan non-linear antara variabel prediktor x dan variabel respons y menggunakan kombinasi potongan polinomial terhubung pada simpul-simpul tertentu (*knots*), disimbolkan dengan $\xi_1 < \xi_2 < \cdots < \xi_K$. Berbeda dari interpolasi yang menyelesaikan SPL deterministik untuk menghubungkan turunan pada setiap titik data, pendekatan *splines regression* mentranslasikan permasalahan ke dalam Sistem Persamaan Linear *Overdetermined* (jumlah data $N >$ jumlah basis M) melalui kerangka ruang fungsi basis, kemudian diselesaikan menggunakan Metode Kuadrat Terkecil (*Ordinary Least Squares*).

1. Konstruksi Ruang Fungsi dan Basis Daya Terpancung (*Truncated Power Basis*)

Ruang fungsi spline kubik dengan K simpul internal $\xi_1, \xi_2, \dots, \xi_K$ merupakan ruang vektor berdimensi $M = 4 + K$, di mana 4 adalah jumlah koefisien polinomial kubik standar dan K adalah jumlah fungsi basis tambahan untuk setiap simpul.

Setiap fungsi $f(x)$ dalam ruang tersebut dapat dinyatakan sebagai kombinasi linear unik dari M buah fungsi basis $\{h_0(x), h_1(x), h_2(x), h_3(x), \dots, h_{3+K}(x)\}$:

Polinomial standar global:

$$h_0(x) = 1, \quad h_1(x) = x, \quad h_2(x) = x^2, \quad h_3(x) = x^3$$

Basis tambahan di setiap simpul ξ_k:

$$h_{3+k}(x) = (x - \xi_k)_+^3 = \begin{cases} (x-\xi_k)^3, & \text{jika } x > \xi_k \\ 0, & \text{jika } x \le \xi_k \end{cases} \quad \text{untuk } k = 1, 2, \dots, K$$

Fungsi regresi menjadi:

$$f(x) = \sum_{j=0}^{3+K} \beta_j h_j(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \sum_{k=1}^{K} \beta_{3+k} (x - \xi_k)_+^3$$

Sifat kontinuitas turunan hingga C^2 di seluruh rentang x langsung terpenuhi secara analitis oleh bentuk fungsi basis $(x - \xi_k)_+^3$ tanpa memerlukan penambahan persamaan batas turunan manual.

Catatan notasi:

$$(u)_+ = \max(0, u) = \begin{cases} u, & \text{jika } u > 0 \\ 0, & \text{jika } u \leq 0 \end{cases}$$

2. Formulasi Matriks Desain ($X\boldsymbol{\beta} \approx \mathbf{y}$)

Diberikan N sampel data pengamatan $(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$ dengan kondisi jumlah data jauh lebih banyak daripada jumlah basis ($N > M$). Persamaan untuk setiap titik observasi $i = 1, 2, \ldots, N$ dituliskan sebagai:

$$y_i = \beta_0 h_0(x_i) + \beta_1 h_1(x_i) + \cdots + \beta_{3+K} h_{3+K}(x_i) + \varepsilon_i$$

Dalam notasi matriks-vektor aljabar linear:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

dengan:

- Vektor observasi: $\mathbf{y} = \begin{bmatrix} y_1 & y_2 & \ldots & y_N \end{bmatrix}^T \in \mathbb{R}^{N \times 1}$
- Vektor parameter bobot: $\boldsymbol{\beta} = \begin{bmatrix} \beta_0 & \beta_1 & \ldots & \beta_{3+K} \end{bmatrix}^T \in \mathbb{R}^{M \times 1}$
- Matriks Desain $X \in \mathbb{R}^{N \times M}$:

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 & (x_1 - \xi_1)_+^3 & (x_1 - \xi_2)_+^3 & \cdots & (x_1 - \xi_K)_+^3 \\ 1 & x_2 & x_2^2 & x_2^3 & (x_2 - \xi_1)_+^3 & (x_2 - \xi_2)_+^3 & \cdots & (x_2 - \xi_K)_+^3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & x_N^3 & (x_N - \xi_1)_+^3 & (x_N - \xi_2)_+^3 & \cdots & (x_N - \xi_K)_+^3 \end{bmatrix}$$

3. Proyeksi Ortogonal dan Persamaan Normal

Karena sistem $X\boldsymbol{\beta} = \mathbf{y}$ bersifat *overdetermined* (baris $N >$ kolom M), vektor respons $\mathbf{y}$ umumnya tidak terletak di dalam ruang kolom matriks X (Col(X)). Solusi terbaik diperoleh dengan mencari vektor taksiran $\hat{\mathbf{y}} = X\hat{\boldsymbol{\beta}}$ yang merupakan proyeksi ortogonal dari $\mathbf{y}$ ke subruang Col(X), meminimalkan norma kuadrat vektor residu $\|\mathbf{e}\|^2$:

$$\min_{\boldsymbol{\beta}} \|\mathbf{y} - X\boldsymbol{\beta}\|^2 = (\mathbf{y} - X\boldsymbol{\beta})^T(\mathbf{y} - X\boldsymbol{\beta})$$

Berdasarkan teorema proyeksi aljabar linear, vektor sisaan $\mathbf{e} = \mathbf{y} - X\hat{\boldsymbol{\beta}}$ harus tegak lurus (ortogonal) terhadap seluruh kolom X:

$$X^T(\mathbf{y} - X\hat{\boldsymbol{\beta}}) = \mathbf{0}$$

Persamaan ini menghasilkan Persamaan Normal (*Normal Equations*):

$$(X^T X)\hat{\boldsymbol{\beta}} = X^T\mathbf{y}$$

Estimasi Parameter Koefisien

Jika kolom-kolom pada matriks X bebas linear (*linearly independent*), matriks Gram $X^T X$ berukuran $M \times M$ bersifat simetris definit positif dan memiliki invers, sehingga solusi parameter koefisien $\hat{\boldsymbol{\beta}}$ ditentukan secara unik:

$$\hat{\boldsymbol{\beta}} = (X^T X)^{-1} X^T \mathbf{y}$$

Penjelasan lebih mendalam mengenai formulasi matematis, fungsi basis terpancung, serta konsep regularisasi regresi spline dapat dirujuk pada buku *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* karya Hastie, Tibshirani, dan Friedman.

3 Implementasi

Program yang dibangun adalah kalkulator aljabar linier berbasis antarmuka baris perintah (*command line interface*) yang ditulis dalam bahasa Java (versi 21) tanpa memakai pustaka aljabar linier pihak ketiga. Seluruh kode berada di bawah paket induk `algeo` dan dibagi menjadi paket-paket kecil sesuai tanggung jawabnya. Fitur yang tersedia meliputi penyelesaian sistem persamaan linear (SPL), determinan, invers matriks, interpolasi polinomial, interpolasi spline kubik natural, dan regresi spline kubik.

3.1 Struktur Kelas

Bagian ini menjelaskan kelas-kelas yang didefinisikan beserta atribut dan metode pentingnya, dikelompokkan menurut paket. Kelas utilitas (`DeterminantOBE`, `DeterminantCofactor`, `InverseAugmented`, `InverseAdjoint`, `PointSetUtils`, dan `IOUtils`) memiliki konstruktor `private` sehingga tidak dapat diinstansiasi dan seluruh metodenya bersifat `static`. Konstanta toleransi numerik $\varepsilon = 10^{-9}$ dipakai secara konsisten (atribut `EPSILON`) untuk membandingkan bilangan `double` dengan nol.

3.1.1 Paket `algeo.matrix`

Tabel 3.1: Kelas pada paket `algeo.matrix`

Kelas	Atribut penting	Metode penting
`Matrix`	`int rows`, `int cols`, `double[][] spl`, `EPSILON` (konstanta)	Konstruktor: `Matrix(int, int)`, `Matrix(double[][])`; akses elemen: `get`, `set`, `getRow`, `getColumn`, `setRow`, `setColumn`; operasi: `copy`, `swapRows`, `add`, `subtract`, `multiplyByScalar`, `multiply`, `transpose`, `getSubmatrix`, `augment`; bantuan eliminasi: `getPivotRow`, `partialPivot`; `equals` dan `hashCode` yang toleran terhadap ε.

Kelas	Atribut penting	Metode penting
`Matrix Exception`	(mewarisi `RuntimeException`)	Konstruktor: `MatrixException(String message)`. Dipakai sebagai `unchecked exception` untuk seluruh kesalahan domain: dimensi tidak valid, indeks di luar batas, matriks singular, input tidak valid, dan sebagainya.

Seluruh akses ke `spl` divalidasi melalui `validateRowIndex` dan `validateColIndex` sehingga pelanggaran indeks menghasilkan `MatrixException` yang pesannya mudah dibaca.

3.1.2 Paket `algeo.spl`

Paket ini memuat antarmuka `SPLSolver` dan empat implementasinya. Rancangan ini mengikuti pola *Strategy*: `App` hanya bergantung pada antarmuka, sehingga metode penyelesaian dapat dipilih saat program berjalan tanpa percabangan logika di sisi pemanggil.

Tabel 3.2: Kelas pada paket `algeo.spl`

Kelas	Atribut penting	Metode penting
`SPLSolver` (interface)	—	`getName()`, `isApplicable(Matrix)`, `solve(Matrix)` (mengembalikan `String` berisi langkah dan hasil), `solveToVector(Matrix)` (mengembalikan `double[]` bila solusi unik, `null` bila tidak), serta *default method* `solve(Matrix A, Matrix B)`.
`Gauss`	`EPSILON`	Implementasi `SPLSolver`. Metode privat: `forwardElimination` (eliminasi maju dengan *partial pivoting*, mengembalikan rank dan kolom pivot), `isInconsistent`, `formatExpression` (bentuk parametrik $t_1, t_2, \ldots$), `appendAugmentedMatrix`. Solusi dihitung dengan substitusi mundur.
`GaussJordan`	`EPSILON`	Implementasi `SPLSolver`. Metode privat `reduce` melakukan eliminasi maju lalu mundur hingga bentuk eselon baris tereduksi; solusi dibaca langsung dari matriks hasil.

Kelas	Atribut penting	Metode penting
`InverseSPL`	—	`isApplicable` hanya benar bila matriks koefisien persegi. `solve` memeriksa determinan lalu menghitung $x = A^{-1}b$ dengan `InverseAugmented`.
`Cramer`	—	`isApplicable` hanya benar bila matriks koefisien persegi. `solve` menghitung $\det(A)$ dan $\det(A_i)$ dengan `DeterminantOBE`, lalu $x_i = \det(A_i)/\det(A)$.

3.1.3 Paket `algeo.determinant`

Tabel 3.3: Kelas pada paket `algeo.determinant`

Kelas	Atribut	Metode penting (semua `static`)
`DeterminantOBE`	`EPSILON`	`determinantOBE(Matrix)`: reduksi baris ke segitiga atas dengan *partial pivoting*; determinan adalah hasil kali diagonal dengan tanda dibalik pada setiap pertukaran baris. `determinantOBEWithSteps(Matrix)`: versi yang mencatat setiap langkah.
`Determinant Cofactor`	`EPSILON`	`cofactorExpansion(Matrix)`: ekspansi kofaktor rekursif pada baris atau kolom dengan elemen nol terbanyak (`findBestLine`), suku bernilai nol dilewati. `cofactorExpansionWithSteps(Matrix)`: versi berlangkah.

3.1.4 Paket `algeo.inverse`

Tabel 3.4: Kelas pada paket `algeo.inverse`

Kelas	Atribut	Metode penting (semua `static`)
`Inverse Augmented`	`EPSILON`	`inverse(Matrix)`: membentuk $[A \mid I]$ lalu eliminasi Gauss-Jordan dengan *partial pivoting*; `roundForDisplay(Matrix, int)`; `inverseWithSteps(Matrix)`.
`InverseAdjoint`	`EPSILON`	`cofactor`, `cofactorMatrix`, `adjoint`, dan `inverse(Matrix)` berdasarkan $A^{-1} = \frac{1}{\det(A)} \operatorname{adj}(A)$ (determinan dihitung dengan `DeterminantOBE`); `inverseWithSteps(Matrix)`.

3.1.5 Paket `algeo.common`

Tabel 3.5: Kelas pada paket `algeo.common`

Kelas	Atribut	Metode penting
`PointSetUtils`	`EPSILON`	`validateAndSort(double[] x, double[] y, int maxPoints)`: memeriksa `null` dan panjang array, jumlah titik minimal 2 dan maksimal `maxPoints`, mengurutkan titik menaik terhadap x (*selection sort*), dan menolak nilai x duplikat.
`PointSetUtils.SortedPoints` (kelas dalam)	`double[] x`, `double[] y` (`final`)	Wadah data titik yang sudah tervalidasi dan terurut.

3.1.6 Paket `algeo.interpolation` dan `algeo.spline`

Ketiga kelas kurva berikut memanggil `PointSetUtils.validateAndSort` pada konstruktornya, lalu menyimpan koefisien sebagai atribut `final`. Setelah objek dibuat, kurva siap dievaluasi di sembarang titik.

Tabel 3.6: Kelas kurva

Kelas	Atribut penting	Metode penting
`PolynomialInterpolation`	`x`, `y`, `coefficients`, `degree`, `MAX_POINTS` = 10	Konstruktor membentuk matriks augmented Vandermonde, lalu `solveLinearSystem` (Gauss dengan *partial pivoting*). `evaluate(double)` memakai skema Horner. Juga `getDomain`, `getCoefficients`, `getDegree`, `toEquationString`.
`NaturalCubicSpline`	`x`, `y`, `n` (jumlah segmen), koefisien `a`, `b`, `c`, `d`, `MAX_POINTS` = 10	`buildSpline` (algoritma tridiagonal dengan syarat batas natural $S'' = 0$), `evaluate`, `findSegment`, `getSecondDerivatives`, `getSegmentDomain`, `getSegmentEquationString`, `toFullEquationString`.

Kelas	Atribut penting	Metode penting
`CubicSpline Regression`	`x`, `y`, `knots`, `beta`, `gamma`, `center`, `halfRange`, `RANK_TOLERANCE`	Konstruktor memvalidasi titik dan knot serta memastikan $n > M = 4 + K$. `buildRegression` membentuk matriks desain dan menyelesaikan persamaan normal $(X^\top X)\beta = X^\top y$ dengan `solveGaussJordan`. Juga `scaledBasis`, `evaluate`, `getResidualSumOfSquares`, `getKnots`, `toFullEquationString`.

Pada `CubicSplineRegression`, variabel x diskalakan ke bentuk $u = (x - \text{center})/\text{halfRange}$ agar matriks $X^\top X$ lebih stabil secara numerik. Koefisien pada basis terskala (`gamma`) kemudian dikonversi kembali menjadi koefisien `beta` pada variabel x asli untuk ditampilkan.

3.1.7 Paket `algeo.io` dan kelas `App`

Tabel 3.7: Kelas masukan/keluaran dan kelas utama

Kelas	Atribut penting	Metode penting
`IOUtils`	`MAX_MANUAL_ MATRIX_SIZE` = 11, `MAX_FILE_ MATRIX_ROWS` = 1001, `MAX_FILE_ MATRIX_COLUMNS` = 1002, `MAX_CURVE_POINTS` = 10	`parseDouble` (menerima koma desimal dan tanda minus Unicode), `format3`, `matrixToString`, `printMatrix`, `readMatrixManual`, `readAugmentedManual`, `readPointsManual`, `readMatrixFromFile`, `readPointsFromFile`, `saveToFile`, `readInt`. Metode privat: `validatePointCount`, `readNonEmptyLines`.
`App`	`MAX_ INTERPOLATION_ POINTS` = 10	`main` (perulangan menu utama dan penanganan eksepsi), `printMainMenu`, `handleSPL`, `handleDeterminant`, `handleInverse`, `handlePolynomialInterpolation`, `handleNaturalCubicSpline`, `handleCubicSplineRegression`; pembantu: `askInputSource`, `obtainSquareMatrix`, `obtainAugmentedMatrix`, `obtainPoints`, `extrapolationNote`, `pointsToString`, `offerSave`.

3.2 Arsitektur Pustaka

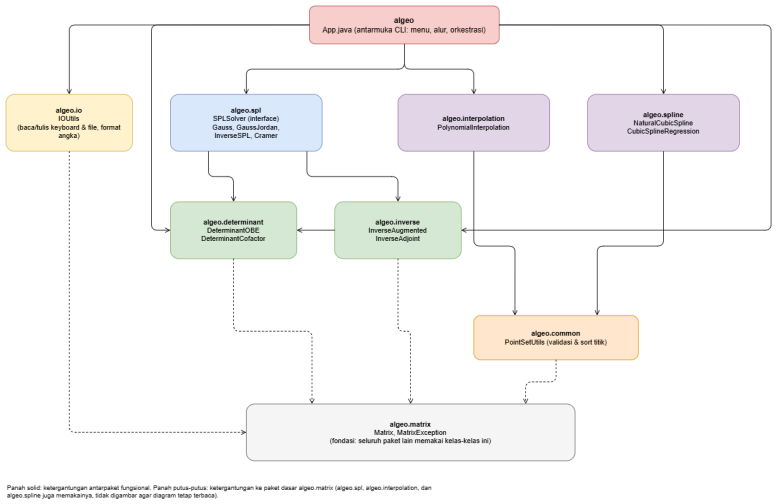
3.2.1 Pembagian Paket

Pustaka disusun berlapis (*layered*). Lapisan paling atas hanya mengurus interaksi dengan pengguna, lapisan tengah berisi algoritma, dan lapisan paling bawah berisi struktur data matriks. Ketergantungan hanya mengalir dari atas ke bawah sehingga tidak terdapat ketergantungan siklik antarpaket.

Tabel 3.8: Tanggung jawab dan ketergantungan setiap paket

Paket	Tanggung jawab	Bergantung pada
`algeo`	Antarmuka CLI, menu, dan orkestrasi alur	seluruh paket di bawah
`algeo.io`	Pembacaan keyboard/berkas dan penyimpanan hasil	`matrix`
`algeo.spl`	Penyelesaian SPL dengan empat metode	`matrix`, `determinant`, `inverse`
`algeo.determinant`	Determinan (OBE dan kofaktor)	`matrix`
`algeo.inverse`	Invers (augmentasi dan adjoin)	`matrix`, `determinant`
`algeo.interpolation`	Interpolasi polinomial	`matrix`, `common`
`algeo.spline`	Spline kubik natural dan regresi spline kubik	`matrix`, `common`
`algeo.common`	Validasi dan pengurutan titik data	`matrix`
`algeo.matrix`	Struktur data matriks dan `MatrixException`	(tidak ada)

Hubungan antarmodul diperlihatkan pada Gambar 3.1.



Gambar 3.1: Arsitektur pustaka dan ketergantungan antarpaket

3.2.2 Prinsip Perancangan

- **Penggunaan ulang kode.** `DeterminantOBE` dipakai oleh `Cramer`, `InverseSPL`, dan `InverseAdjoint`; `InverseAugmented` dipakai oleh `InverseSPL`; `PointSetUtils` dipakai bersama oleh interpolasi polinomial, spline natural, dan regresi spline.
- **Pola *Strategy*.** Keempat metode SPL mengimplementasikan `SPLSolver`, sehingga `App` cukup memilih objek yang sesuai lalu memanggil `isApplicable` dan `solve`.
- **Pemisahan logika dan tampilan.** Kelas algoritma tidak mencetak ke layar; hasil dikembalikan sebagai `double`, objek kurva, atau `String` langkah penyelesaian, dan `App`-lah yang mencetak serta menyimpannya. Karena itu pustaka dapat dipakai ulang oleh antarmuka lain.
- **Penanganan kesalahan terpusat.** Seluruh pelanggaran domain dilaporkan sebagai `MatrixException` dan ditangkap di `App.main`, sehingga program tidak berhenti akibat masukan salah.
- **Ketahanan data.** Metode `solve` bekerja pada salinan matriks (`copy()`) agar masukan pengguna tidak berubah, dan metode *getter* array mengembalikan hasil `clone()`.

3.2.3 Struktur Direktori Proyek

Proyek mengikuti tata letak standar Maven; nama paket Java sama dengan struktur foldernya.

```
algeo26-tb1-Padang/
|-- .vscode
    |-- settings.json
|-- bin
    |-- .gitkeep/
    |-- matrix-calculator-1.0-SNAPSHOT.jar
|-- docs
    |-- .gitkeep/
|-- src/main/
    |-- java/algeo/
    |   |-- App.java
    |   |-- GuiApp.java
    |   |-- common/
    |   |   `-- PointSetUtils.java
    |   |-- determinant/
    |   |   |-- DeterminantCofactor.java
    |   |   `-- DeterminantOBE.java
    |   |-- interpolation/
    |   |   `--PolynomialInterpolation.java
    |   |-- inverse/
    |   |   |-- InverseAdjoint.java
    |   |   `-- InverseAugmented.java
    |   |-- io/
    |   |   `-- IOUtils.java
```

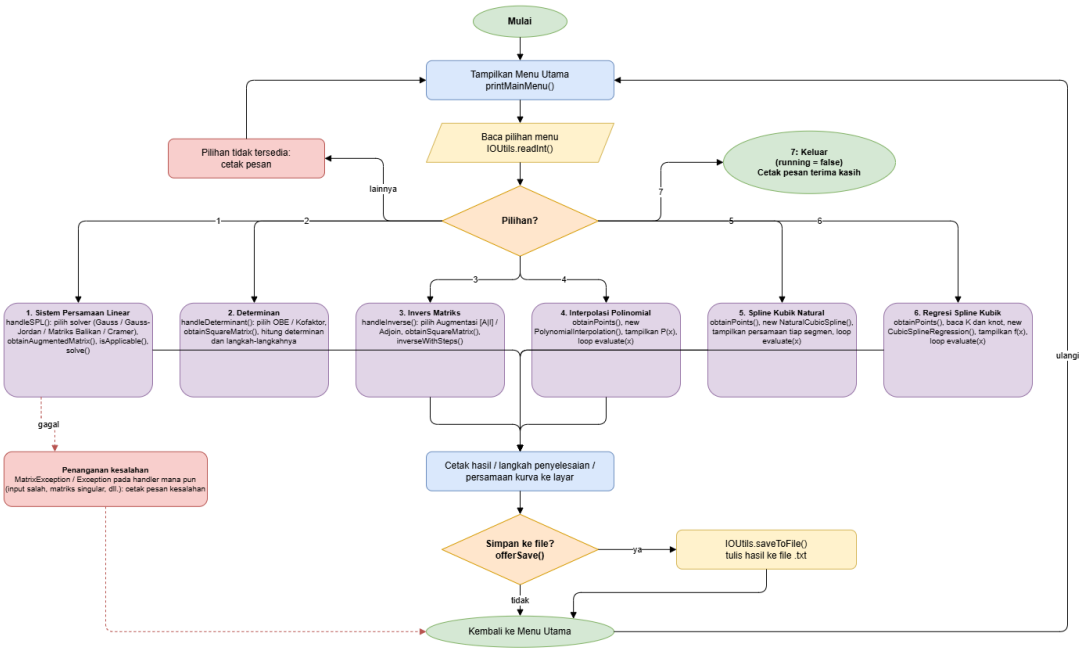
```
|   |-- matrix/
|   |   |-- Matrix.java
|   |   `-- MatrixException.java
|   |-- spl/
|   |   |-- SPLSolver.java
|   |   |-- Gauss.java
|   |   |-- GaussJordan.java
|   |   |-- InverseSPL.java
|   |   `-- Cramer.java
|   `-- spline/
|       |-- NaturalCubicSpline.java
|       `-- CubicSplineRegression.java
`-- resources
    |-- algeo/gui/
    |   `-- fonts/
    |       |-- Poppins-Bold.ttf
    |       |-- Poppins-Regular.ttf
    |       `-- Poppins-SemiBold.ttf
    |-- itb + stei.png
    |-- itb.png
    `-- stei.png
|-- test
|-- .gitignore
|-- README.md
`-- pom.xml
```

3.3 Alur Program

Alur program secara garis besar ditunjukkan pada Gambar 3.2.

3.3.1 Alur Utama

1. Program dimulai dari `App.main`, yang membuat objek `Scanner` untuk membaca masukan dari keyboard.
2. Selama `running` bernilai `true`, program menampilkan menu utama (`printMainMenu`) dan membaca pilihan pengguna dengan `IOUtils.readInt`.
3. Pilihan diteruskan ke `handler` yang sesuai: 1 SPL, 2 determinan, 3 invers, 4 interpolasi polinomial, 5 spline kubik natural, 6 regresi spline kubik. Pilihan 7 mengakhiri perulangan dan program berhenti; angka lain menghasilkan pesan bahwa pilihan tidak tersedia.
4. Setiap *handler* meminta masukan (dari keyboard atau berkas `.txt`), memanggil kelas perhitungan, mencetak hasil ke layar, dan menawarkan penyimpanan hasil ke berkas `.txt` melalui `offerSave`.
5. Bila terjadi `MatrixException` (misalnya matriks singular atau format berkas sa-



Gambar 3.2: Alur program secara garis besar

lah) atau `Exception` lain, pesan kesalahan dicetak dan program kembali ke menu utama, bukan berhenti.

6. Setelah selesai, program kembali ke langkah 2.

3.3.2 Pemilihan Sumber Data

Untuk semua fitur, `askInputSource` menawarkan dua sumber data: (1) diketik langsung lewat keyboard, atau (2) dibaca dari berkas `.txt`. Masukan manual dibatasi maksimal 11 baris dan 12 kolom untuk matriks augmented, sedangkan berkas dibatasi 1001 baris dan 1002 kolom. Untuk data titik, jumlah titik dibatasi maksimal 10. Berkas yang mengandung baris kosong di tengah atau jumlah kolom yang tidak seragam ditolak dengan pesan kesalahan.

3.3.3 Alur Sistem Persamaan Linear

1. Pengguna memilih metode (Gauss, Gauss-Jordan, Matriks Balikan, atau Kaidah Cramer); `App` membuat objek `SPLSolver` yang sesuai.
2. Matriks augmented $[A \mid b]$ dibaca melalui `obtainAugmentedMatrix`.
3. `isApplicable` diperiksa. Metode Matriks Balikan dan Cramer hanya berlaku bila jumlah persamaan sama dengan jumlah variabel.
4. `solve` dijalankan dan mengembalikan `String` berisi langkah dan hasil.

Rincian tiap metode:

- **Gauss**: eliminasi maju dengan *partial pivoting*, normalisasi pivot menjadi 1, dan penghapusan elemen di bawah pivot. Baris $[0\ 0\ \cdots\ 0 \mid k]$ dengan $k \neq 0$ menandakan SPL tidak memiliki solusi. Bila konsisten, variabel tanpa pivot menjadi parameter $t_1, t_2, \ldots$ dan variabel pivot dihitung dengan substitusi mundur.
- **Gauss-Jordan**: sama seperti Gauss, dilanjutkan eliminasi mundur agar elemen di atas pivot juga nol; solusi dibaca langsung dari matriks eselon baris tereduksi.
- **Matriks Balikan**: memeriksa $\det(A) \neq 0$, menghitung A^{-1} lewat $[A \mid I]$, lalu $x = A^{-1}b$.
- **Kaidah Cramer**: menghitung $\det(A)$; bila bukan nol, kolom ke-i diganti vektor b untuk memperoleh A_i, dan $x_i = \det(A_i)/\det(A)$.

3.3.4 Alur Determinan dan Invers

Pengguna memilih metode, program membaca matriks persegi melalui `obtainSquare Matrix` (menolak matriks non-persegi), lalu memanggil metode `...WithSteps` untuk mendapatkan langkah penyelesaian. Determinan OBE mereduksi matriks menjadi segitiga atas dan mengalikan diagonalnya; determinan kofaktor mengekspansi baris atau kolom dengan nol terbanyak secara rekursif. Invers augmentasi mereduksi $[A \mid I]$ menjadi $[I \mid A^{-1}]$; invers adjoin menghitung $\frac{1}{\det(A)}\operatorname{adj}(A)$. Bila matriks singular, dilempar `MatrixException` dan pengguna diberi tahu bahwa invers tidak ada.

3.3.5 Alur Interpolasi dan Regresi

1. Titik data dibaca melalui `obtainPoints`. Untuk regresi, jumlah knot K dan posisi setiap knot juga dibaca.
2. Konstruktor kelas kurva memanggil `PointSetUtils.validateAndSort` (2–10 titik, terurut menaik, tanpa x duplikat).
3. Koefisien dihitung:
   - *Interpolasi polinomial*: menyelesaikan sistem Vandermonde derajat $n-1$ dengan eliminasi Gauss.
   - *Spline kubik natural*: menyelesaikan sistem tridiagonal untuk c_j dengan $S''(x_0) = S''(x_n) = 0$, lalu menurunkan b_j dan d_j.
   - *Regresi spline kubik*: knot divalidasi (berada di dalam rentang data, tidak duplikat), matriks desain X dibentuk dari basis $1, x, x^2, x^3, (x-\xi_j)_+^3$, lalu persamaan normal $(X^\top X)\beta = X^\top y$ diselesaikan dengan Gauss-Jordan.
4. Program mencetak rentang data dan persamaan kurva, lalu memasuki perulangan evaluasi: pengguna memasukkan nilai x, program mencetak nilai fungsi, dan bila x di luar rentang data ditambahkan peringatan ekstrapolasi. Perulangan berhenti saat pengguna menekan Enter pada baris kosong.
5. Hasil, persamaan kurva, dan catatan evaluasi dapat disimpan ke berkas `.txt`.

3.4 Build Tool dan Packaging

3.4.1 Maven sebagai Build Tool

Proyek ini memakai Apache Maven untuk mengelola kompilasi, dependensi, dan pembuatan berkas `.jar`. Konfigurasi proyek dituliskan pada berkas `pom.xml` di direktori utama, dengan isi pokok sebagai berikut.

- **Koordinat proyek**: `groupId` = `algeo`, `artifactId` = `matrix-calculator`, `version` = `1.0-SNAPSHOT`.
- **Properti**: kompilator diatur ke Java 21 (`maven.compiler.source` dan `maven.compiler.target`), pengkodean sumber UTF-8, dan `javafx.version` = 21.0.5.
- **Dependensi**: `javafx-controls` dan `javafx-fxml` dari `org.openjfx`, yang diunduh otomatis dari Maven Central. Versi pada properti `javafx.version` dipakai bersama oleh keduanya.
- **Plugin**: `exec-maven-plugin` (3.1.0) dengan `mainClass` = `algeo.App` untuk menjalankan versi CLI, dan `javafx-maven-plugin` (0.0.8) untuk menjalankan versi GUI.

Maven bekerja dengan *build lifecycle*. Fase yang paling sering dipakai adalah `compile` (mengompilasi kode di `src/main/java` ke `target/classes`), `test`, `package` (membuat `.jar` di folder `target`), dan `install` (menyalin artefak ke repositori lokal). Menjalankan suatu fase otomatis menjalankan fase-fase sebelumnya.

3.4.2 Perintah Build dan Menjalankan Program

```
# membersihkan hasil build lama dan mengompilasi
mvn clean compile

# menjalankan program CLI langsung dari Maven (kelas utama
algeo.App)
mvn exec:java

# membuat berkas .jar
mvn clean package
```

Perintah `mvn clean package` menghasilkan berkas `target/matrix-calculator-1.0-SNAPSHOT.jar`. Nama berkas ini berasal dari `artifactId` dan `version` yang didefinisikan pada `pom.xml`.

Perintah `mvn clean package` menghasilkan berkas `target/matrix-calculator-1.0-SNAPSHOT.jar`. Nama berkas berasal dari `artifactId` dan `version` pada `pom.xml`.

3.4.3 Pembuatan Berkas .jar yang Dapat Dieksekusi

Secara bawaan, *jar* yang dibuat Maven belum memiliki atribut `Main-Class` pada `MANIFEST.MF`, sehingga tidak dapat dijalankan dengan `java -jar`. Agar dapat dieksekusi langsung, ditambahkan `maven-jar-plugin` pada bagian `<build><plugins>` di `pom.xml`:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-jar-plugin</artifactId>
    <version>3.4.1</version>
    <configuration>
        <archive>
            <manifest>
                <mainClass>algeo.App</mainClass>
            </manifest>
        </archive>
    </configuration>
</plugin>
```

Setelah konfigurasi `pom.xml` diperbarui, jalankan perintah `mvn clean package`. Perintah ini akan menghapus hasil build sebelumnya, mengompilasi kode, dan mengemas proyek. Hasil build akan tersimpan di dalam direktori `target/`. Berkas *jar* tersebut kemudian dapat dijalankan secara langsung dengan perintah:

```
java -jar target/matrix-calculator-1.0-SNAPSHOT.jar
```

Versi CLI (`algeo.App`) hanya memakai pustaka standar Java sehingga *jar* tersebut dapat berjalan tanpa perlu menyertakan JavaFX. Berkas *jar* ini dapat dijalankan pada komputer mana pun yang telah memasang JDK atau JRE versi 21 atau lebih baru.

Untuk menjalankan versi antarmuka grafis (GUI) berbasis JavaFX, pengguna dapat menggunakan perintah `mvn javafx:run`. Jika kelak diperlukan *jar* yang membundel seluruh dependensi eksternal (termasuk JavaFX), dapat digunakan `maven-shade-plugin`.

4 Eksperimen

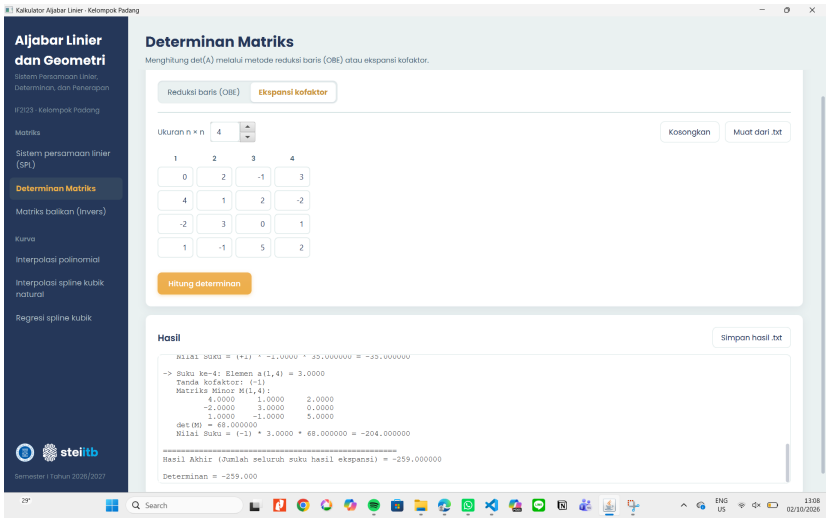
4.1 Kasus Uji 4.1: Determinan Matriks

4.1.1 Kasus Uji 4.1.1: Matriks A Berukuran 4×4

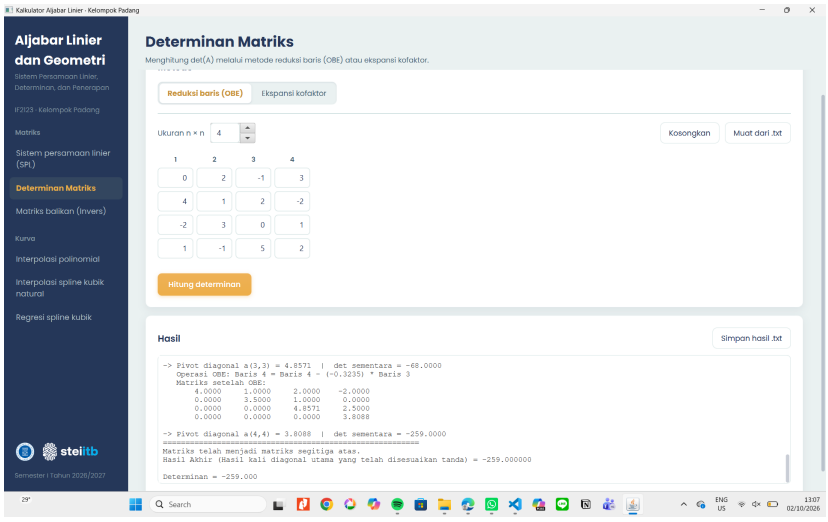
Matriks Masukan:

$$A = \begin{bmatrix} 0 & 2 & -1 & 3 \\ 4 & 1 & 2 & -2 \\ -2 & 3 & 0 & 1 \\ 1 & -1 & 5 & 2 \end{bmatrix}$$

Hasil Eksekusi Program:



Gambar 4.1: Determinan dengan Metode Ekspansi Kofaktor



Gambar 4.2: Determinan dengan Metode Reduksi Baris (OBE)

Analisis:

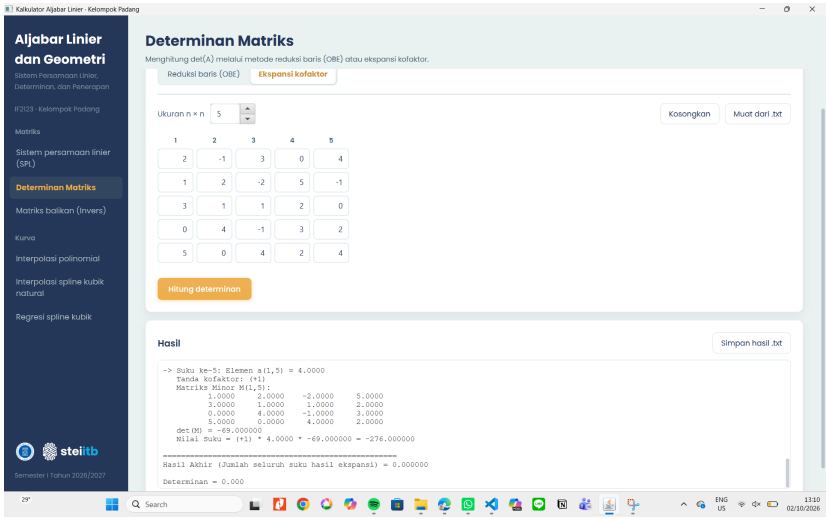
Metode Reduksi Baris (OBE) dan Ekspansi Kofaktor menghasilkan nilai determinan yang sama, yaitu $\det(A)$. Penggunaan *partial pivoting* pada Reduksi Baris membantu menjaga stabilitas numerik ketika terjadi pertukaran baris.

4.1.2 Kasus Uji 4.1.2: Matriks A Berukuran 5×5

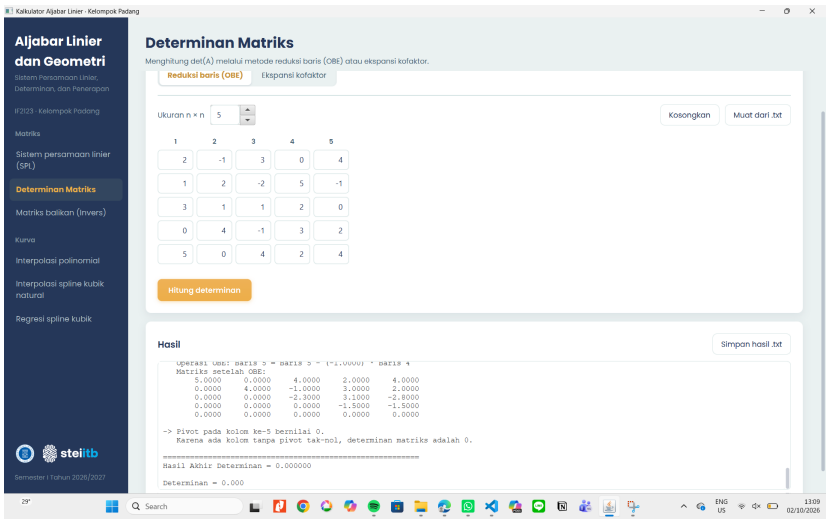
Matriks Masukan:

$$A = \begin{bmatrix} 2 & -1 & 3 & 0 & 4 \\ 1 & 2 & -2 & 5 & -1 \\ 3 & 1 & 1 & 2 & 0 \\ 0 & 4 & -1 & 3 & 2 \\ 5 & 0 & 4 & 2 & 4 \end{bmatrix}$$

Hasil Eksekusi Program:



Gambar 4.3: Determinan dengan Metode Ekspansi Kofaktor



Gambar 4.4: Determinan dengan Metode Reduksi Baris (OBE)

Analisis:

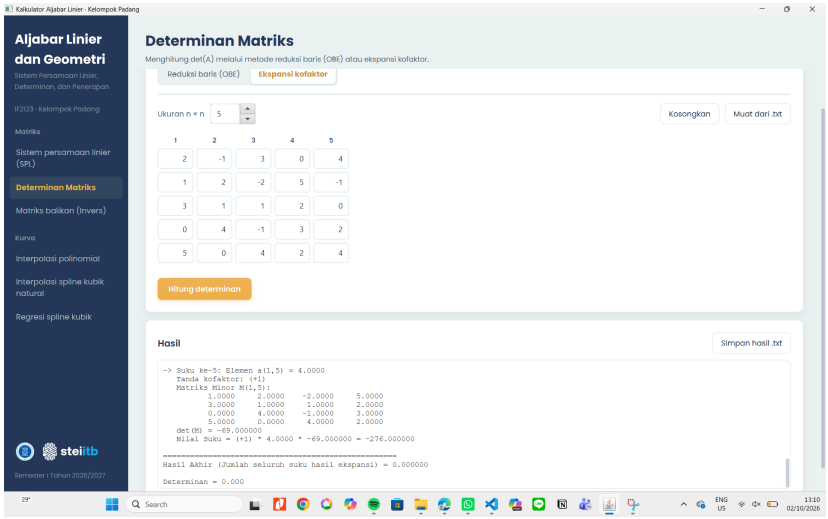
Metode Reduksi Baris (OBE) dan Ekspansi Kofaktor menghasilkan nilai determinan yang sama, yaitu $\det(A)$. Pada matriks berukuran 5×5, Metode Reduksi Baris memiliki efisiensi komputasi yang lebih baik dibandingkan Ekspansi Kofaktor karena kompleksitas algoritmanya yang lebih rendah.

4.1.3 Kasus Uji 4.1.3: Matriks A Berukuran 4×4

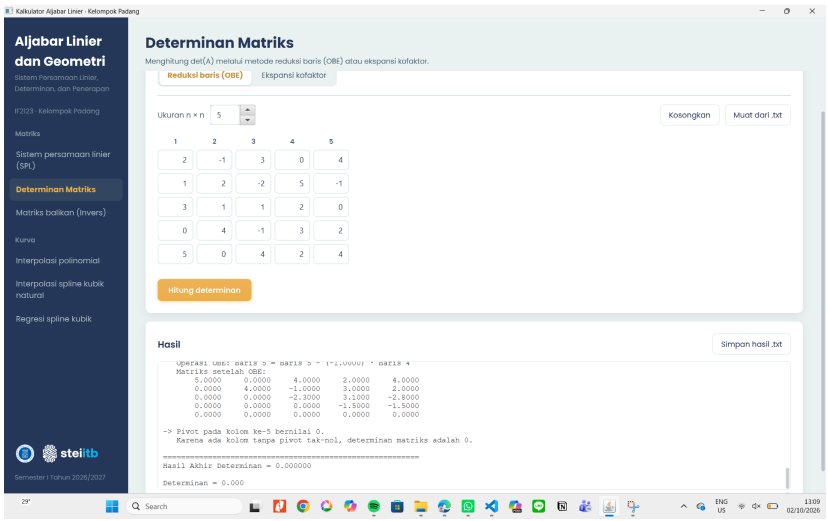
Matriks Masukan:

$$A = \begin{bmatrix} 0.5 & 1 & 0 & 0.5 \\ 0.5 & 1.25 & 0.25 & 0.5 \\ -0.1 & -0.4 & 0.2 & -0.3 \\ 0.6 & 1.25 & 0.45 & 0.6 \end{bmatrix}$$

Hasil Eksekusi Program:



Gambar 4.5: Determinan dengan Metode Ekspansi Kofaktor



Gambar 4.6: Determinan dengan Metode Reduksi Baris (OBE)

Analisis:

Metode Reduksi Baris (OBE) dan Ekspansi Kofaktor menghasilkan nilai determinan yang sama, yaitu $\det(A)$. Pengujian pada matriks berorde 4×4 dengan nilai elemen berupa bilangan desimal menunjukkan bahwa kedua algoritma dapat menangani kalkulasi dengan presisi *floating-point* secara akurat.

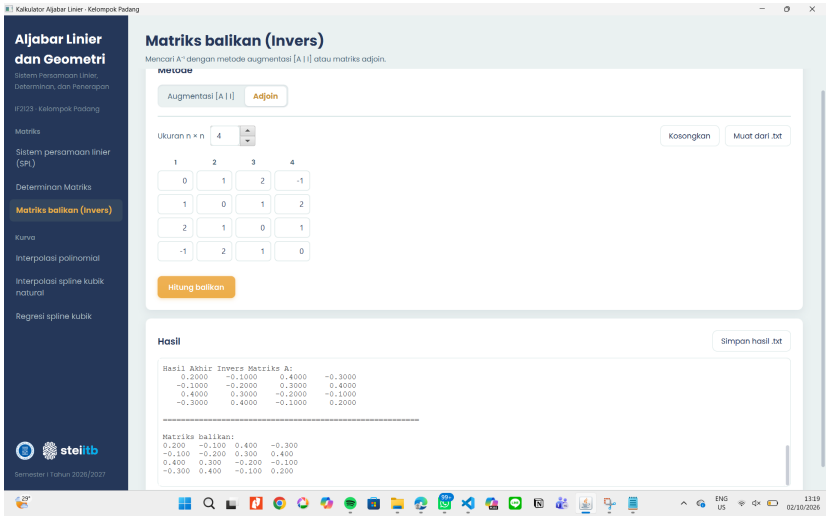
4.2 Kasus Uji 4.2: Invers Matriks

4.2.1 Kasus Uji 4.2.1: Matriks A Berukuran 4×4

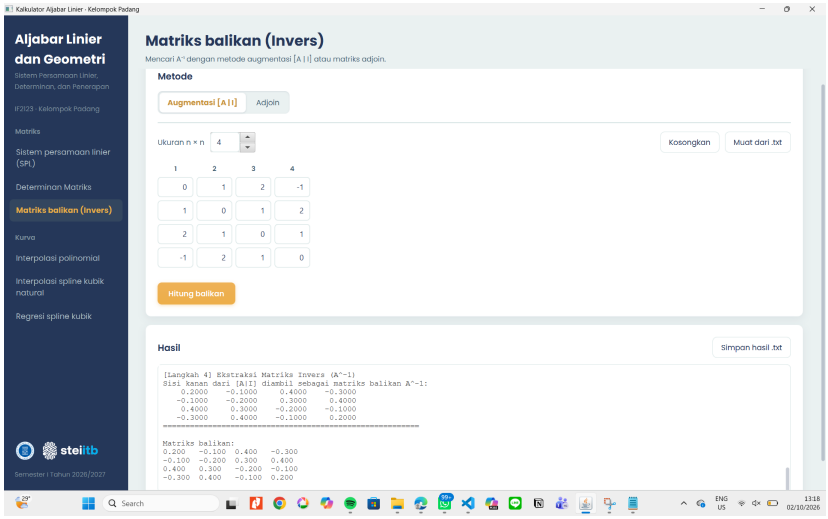
Matriks Masukan:

$$A = \begin{bmatrix} 0 & 1 & 2 & -1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ -1 & 2 & 1 & 0 \end{bmatrix}$$

Hasil Eksekusi Program:



Gambar 4.7: Invers Matriks dengan Metode Adjoin



Gambar 4.8: Invers Matriks dengan Metode Augmentasi (OBE)

Analisis:

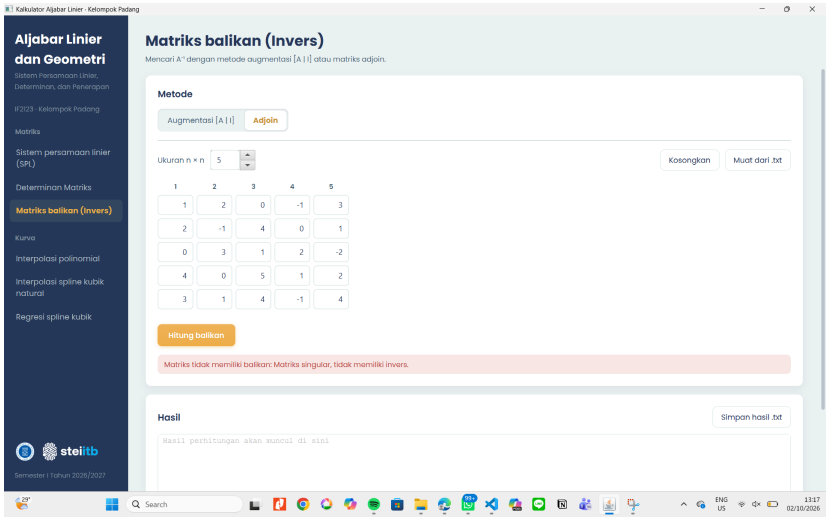
Kedua metode menghasilkan matriks invers yang identik. Penggunaan Metode Augmentasi (OBE) lebih efisien secara komputasi dibandingkan Metode Adjoin, terutama karena Metode Adjoin memerlukan perhitungan kofaktor dan determinan dari banyak submatriks.

4.2.2 Kasus Uji 4.2.2: Matriks A Berukuran 5×5

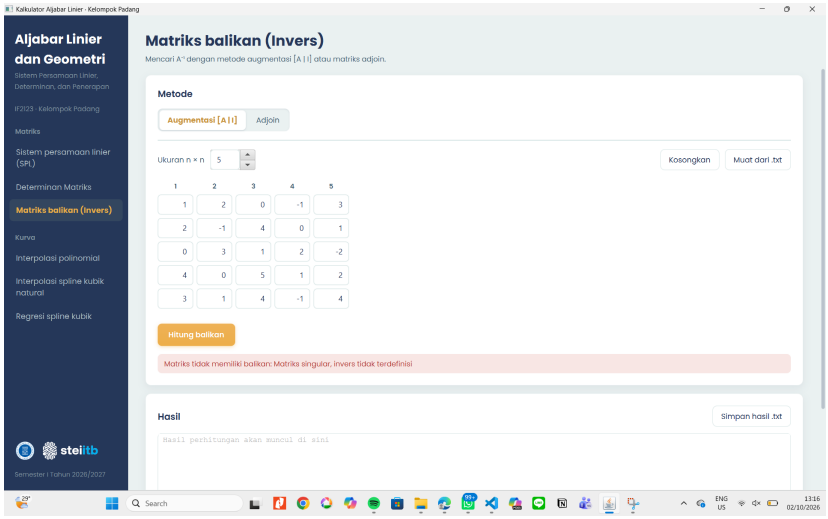
Matriks Masukan:

$$A = \begin{bmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & -1 & 4 & 0 & 1 \\ 0 & 3 & 1 & 2 & -2 \\ 4 & 0 & 5 & 1 & 2 \\ 3 & 1 & 4 & -1 & 4 \end{bmatrix}$$

Hasil Eksekusi Program:



Gambar 4.9: Invers Matriks dengan Metode Adjoin



Gambar 4.10: Invers Matriks dengan Metode Augmentasi (OBE)

Analisis:

Metode Augmentasi dan Adjoin berhasil mendapatkan matriks invers yang sama untuk ukuran 5×5. Terlihat perbedaan performa waktu eksekusi, di mana Metode Augmentasi bekerja lebih efektif daripada Metode Adjoin pada ukuran matriks yang lebih besar.

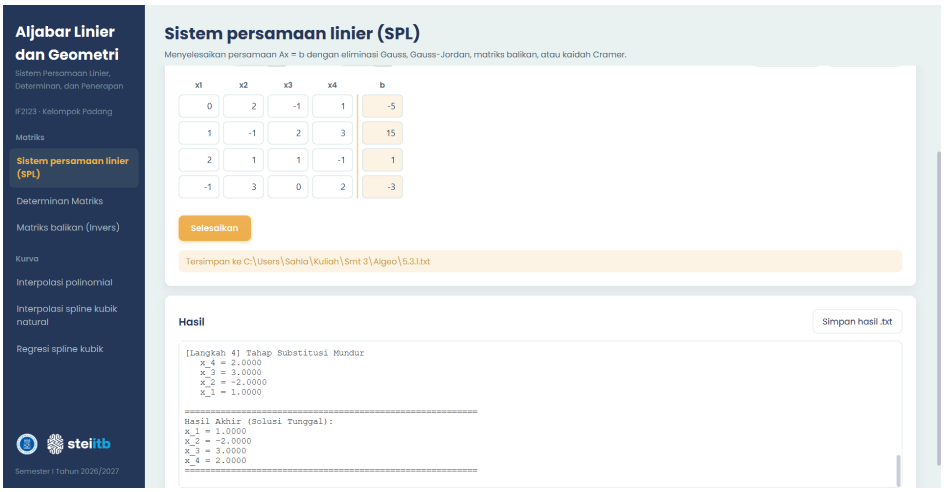
4.3 Kasus Uji 4.3: Sistem Persamaan Linier ($Ax = b$)

4.3.1 Kasus Uji 4.3.1: Matriks A Berukuran 4×4

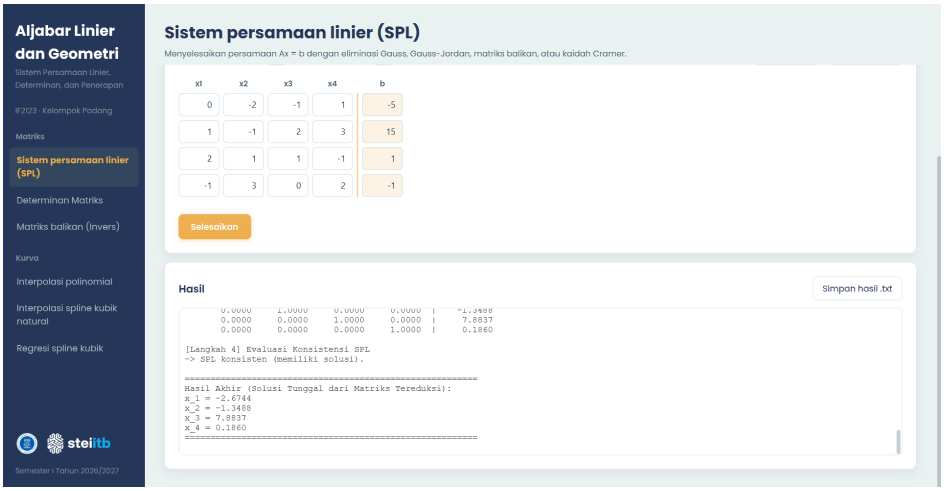
Matriks Masukan:

$$A = \begin{bmatrix} 0 & 2 & -1 & 1 \\ 1 & -1 & 2 & 3 \\ 2 & 1 & 1 & -1 \\ -1 & 3 & 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} -5 \\ 15 \\ 1 \\ -3 \end{bmatrix}$$

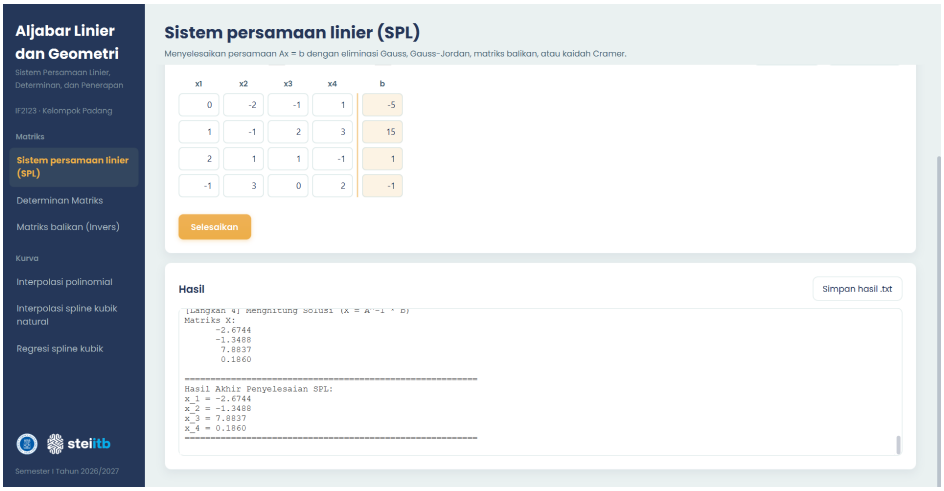
Hasil Eksekusi Program:



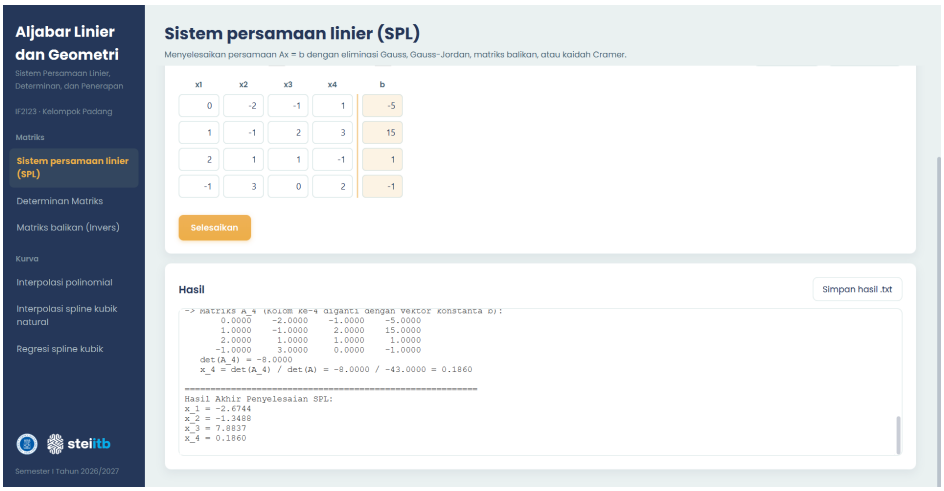
Gambar 4.11: Penyelesaian SPL dengan Metode Gauss



Gambar 4.12: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.13: Penyelesaian SPL dengan Metode Invers



Gambar 4.14: Penyelesaian SPL dengan Kaidah Cramer

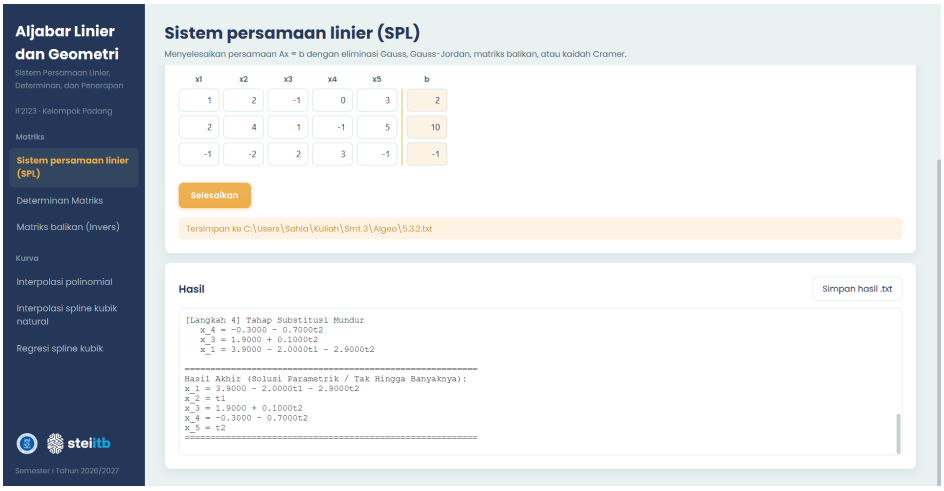
Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan matriks koefisien tidak bernilai nol, matriks memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

4.3.2 Kasus Uji 4.3.2: Matriks A Berukuran 3×5

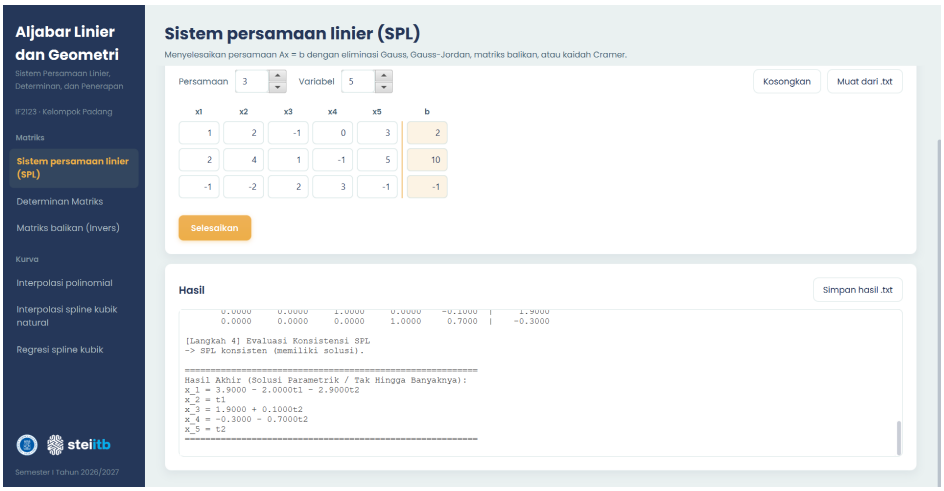
Matriks Masukan:

$$A = \begin{bmatrix} 1 & 2 & -1 & 0 & 3 \\ 2 & 4 & 1 & -1 & 5 \\ -1 & -2 & 2 & 3 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 10 \\ -1 \end{bmatrix}$$

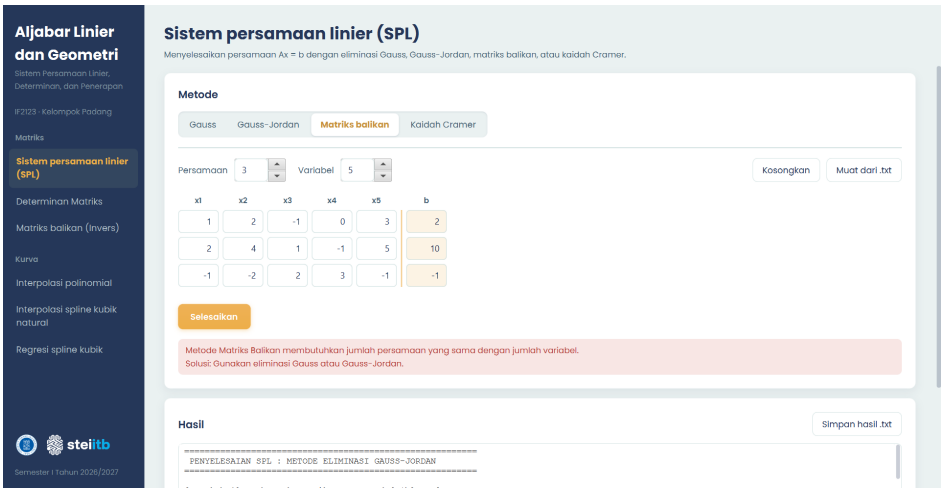
Hasil Eksekusi Program:



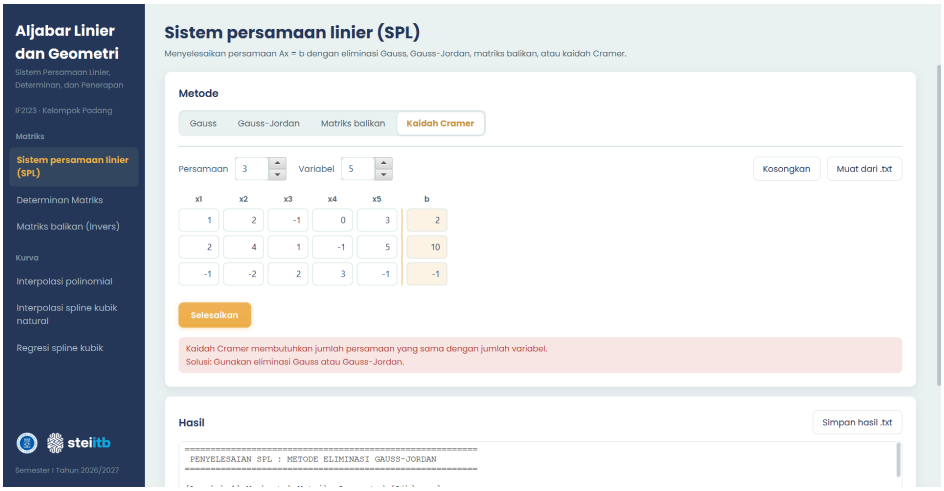
Gambar 4.15: Penyelesaian SPL dengan Metode Gauss



Gambar 4.16: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.17: Penyelesaian SPL dengan Metode Invers



Gambar 4.18: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Matriks A berukuran 3×5, jumlah variabel yang lebih banyak daripada persa-

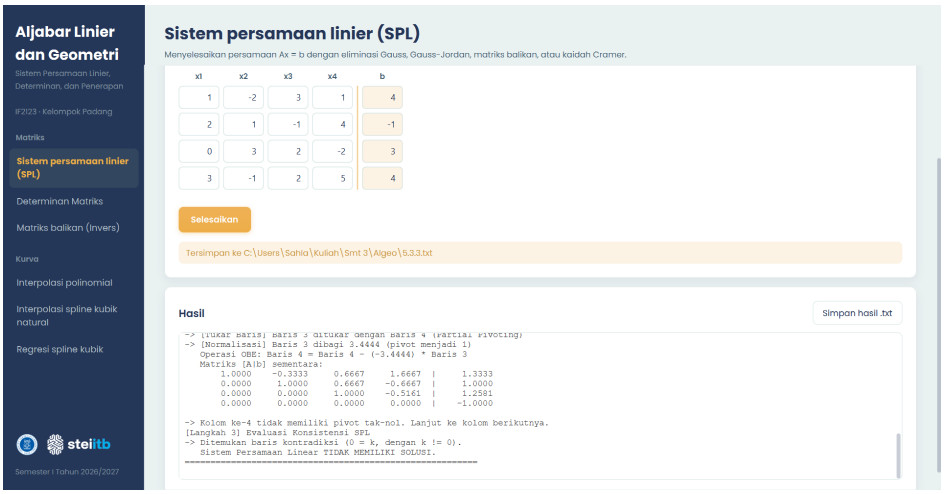
maan tidak memenuhi syarat Kaidah Cramer dan Metode Matriks Balikan sehingga metode yang dapat digunakan hanya metode Eliminasi Gauss dan Gauss-Jordan.

4.3.3 Kasus Uji 4.3.3: Matriks A Berukuran 4×4

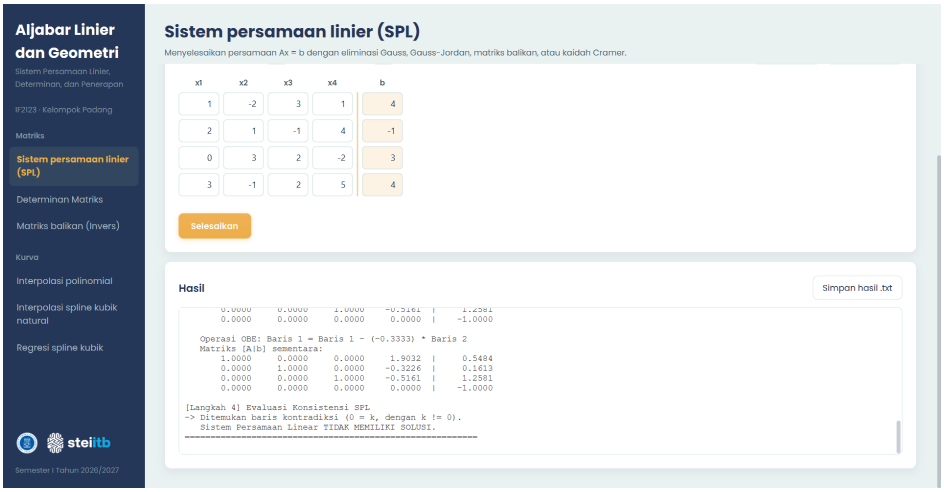
Matriks Masukan:

$$A = \begin{bmatrix} 1 & -2 & 3 & 1 \\ 2 & 1 & -1 & 4 \\ 0 & 3 & 2 & -2 \\ 3 & -1 & 2 & 5 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ -1 \\ 3 \\ 4 \end{bmatrix}$$

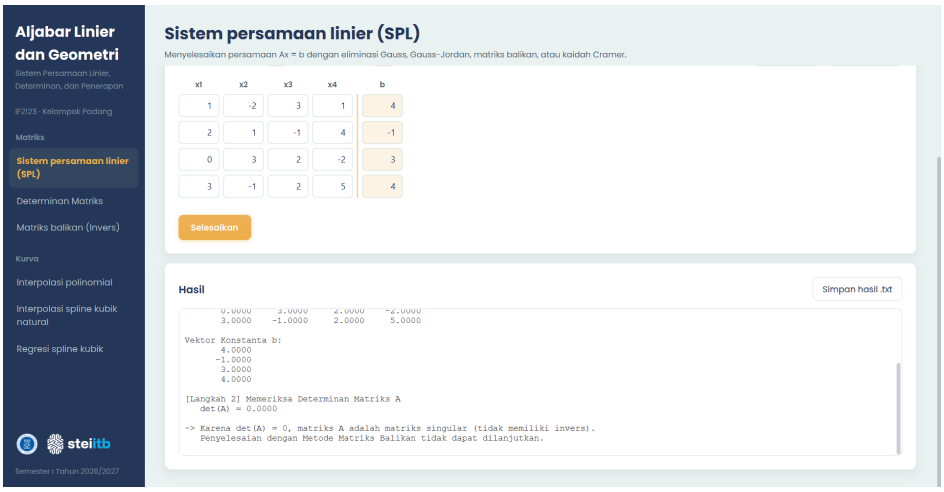
Hasil Eksekusi Program:



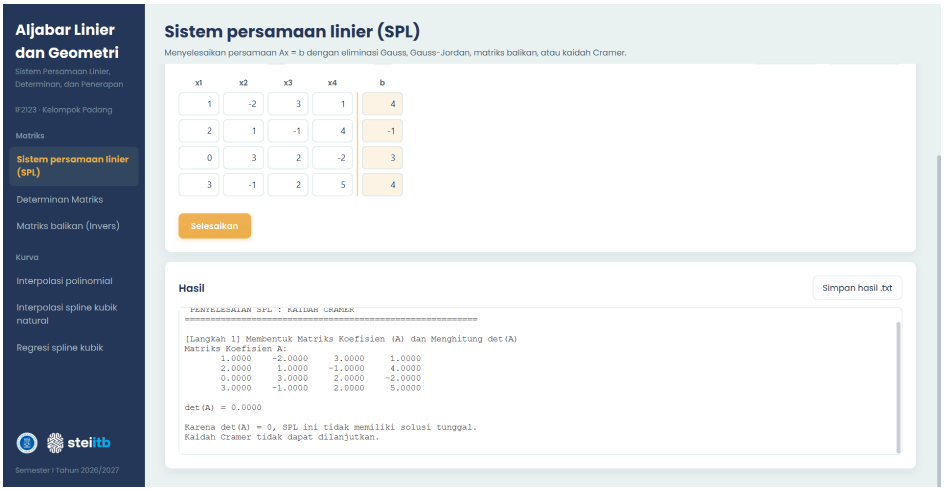
Gambar 4.19: Penyelesaian SPL dengan Metode Gauss



Gambar 4.20: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.21: Penyelesaian SPL dengan Metode Invers



Gambar 4.22: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Berdasarkan hasil perhitungan dari metode Eliminasi Gauss dan Gauss-Jordan, matriks A tidak memiliki solusi dan memiliki $determinan = 0$ sehingga metode Kaidah Cramer dan Metode Matriks Balikan tidak dapat digunakan.

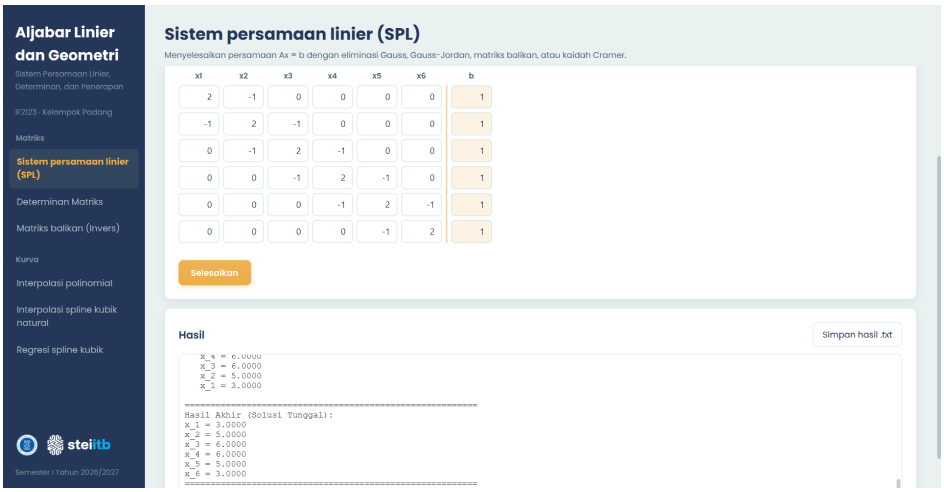
4.3.4 Kasus Uji 4.3.4: Matriks *discrete Laplacian* satu dimensi

Kasus Uji 4.3.4.1: Matriks $n = 6$

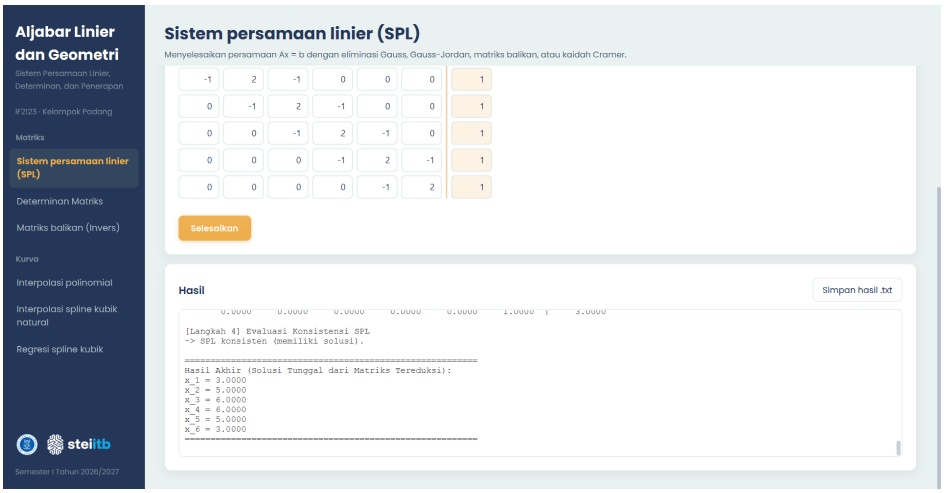
Matriks Masukan:

$$A_6 = \begin{bmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{bmatrix}, \quad b_6 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

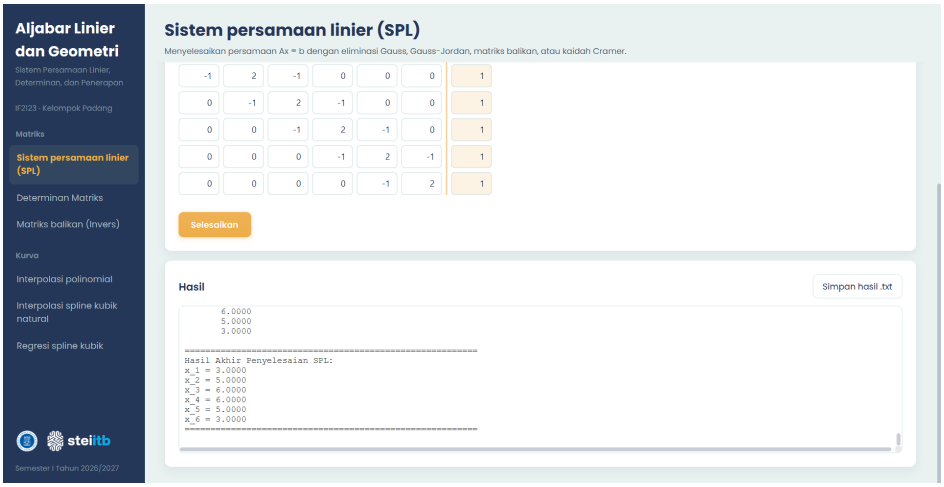
Hasil Eksekusi Program:



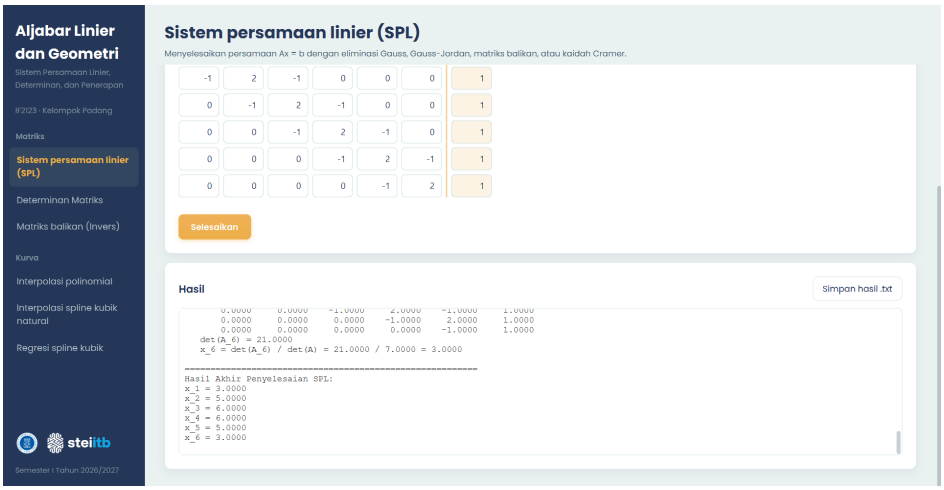
Gambar 4.23: Penyelesaian SPL dengan Metode Gauss



Gambar 4.24: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.25: Penyelesaian SPL dengan Metode Invers



Gambar 4.26: Penyelesaian SPL dengan Kaidah Cramer

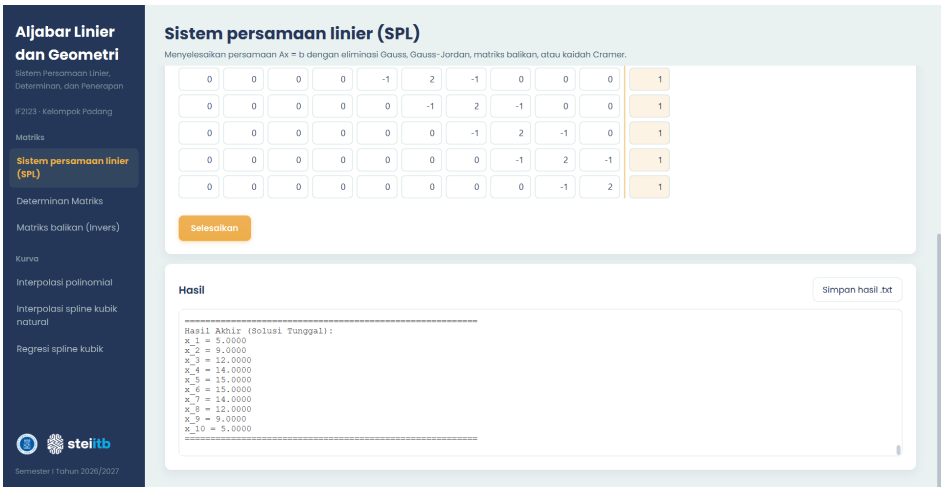
Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan matriks koefisien tidak bernilai nol, matriks memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

Kasus Uji 4.3.4.2: Matriks $n = 10$

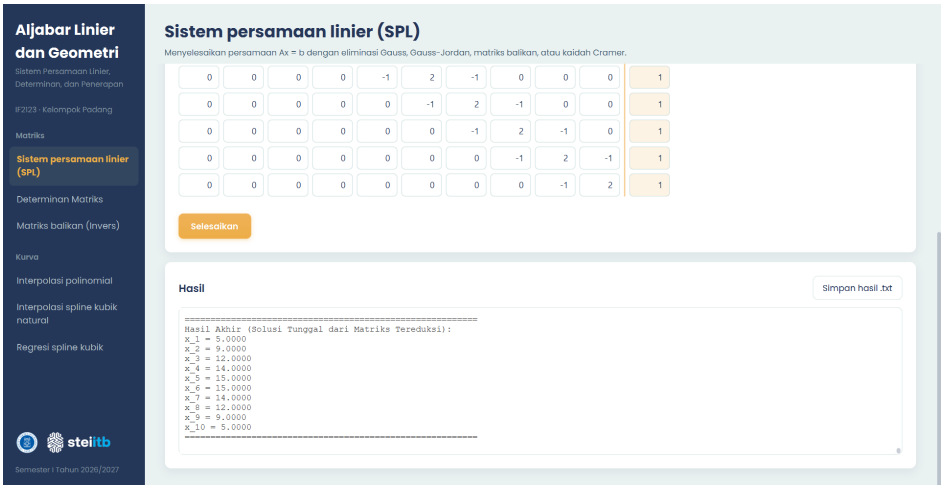
Matriks Masukan:

$$A_{10} = \begin{bmatrix} 2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1 & 2 \end{bmatrix}, \quad b_{10} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

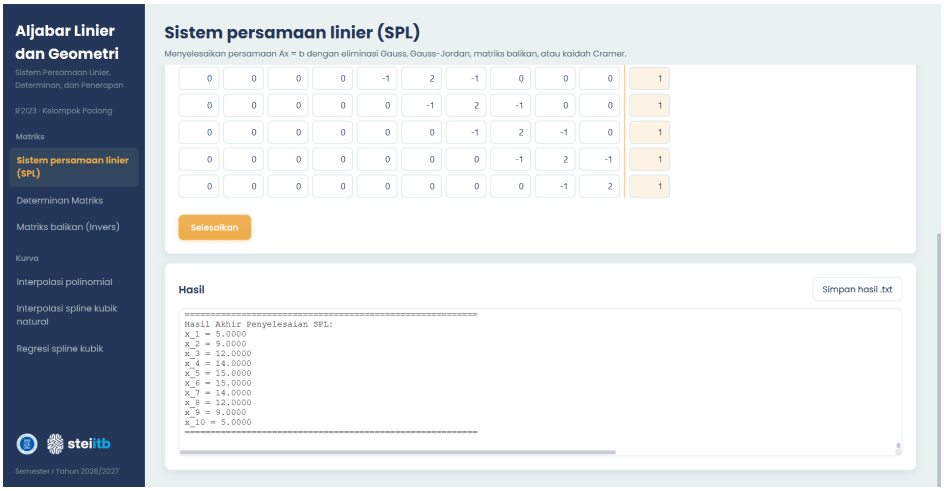
Hasil Eksekusi Program:



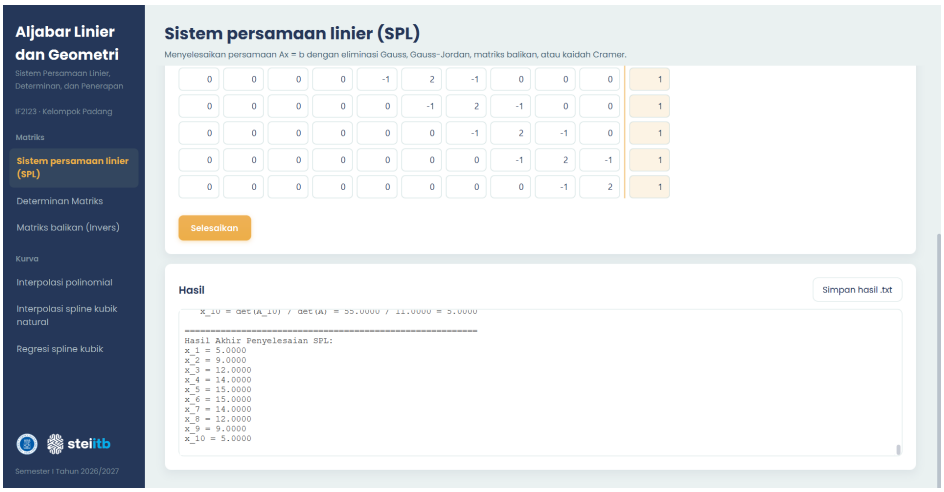
Gambar 4.27: Penyelesaian SPL dengan Metode Gauss



Gambar 4.28: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.29: Penyelesaian SPL dengan Metode Invers



Gambar 4.30: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan matriks koefisien tidak bernilai nol, matriks memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

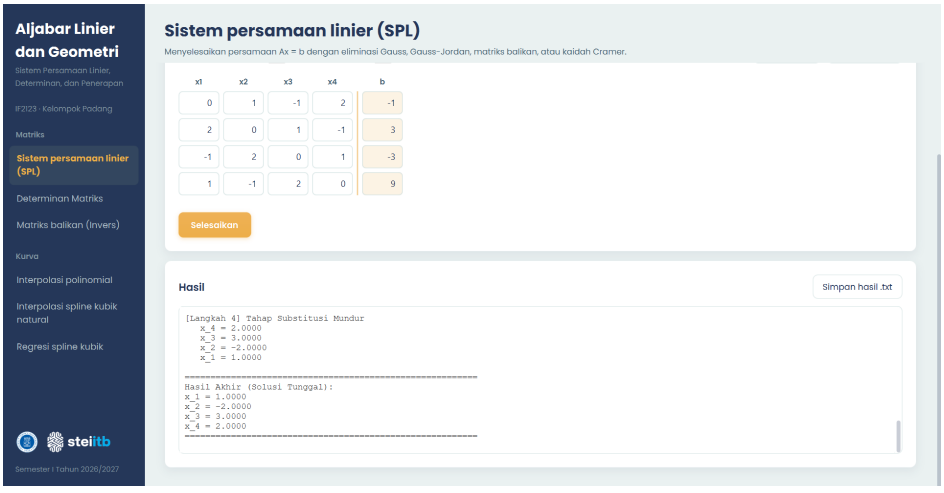
4.4 Kasus Uji 4.4: Sistem Persamaan Linier Berbentuk Matriks Augmented

4.4.1 Kasus Uji 4.4.1: Matriks augmented A Berukuran 4×4

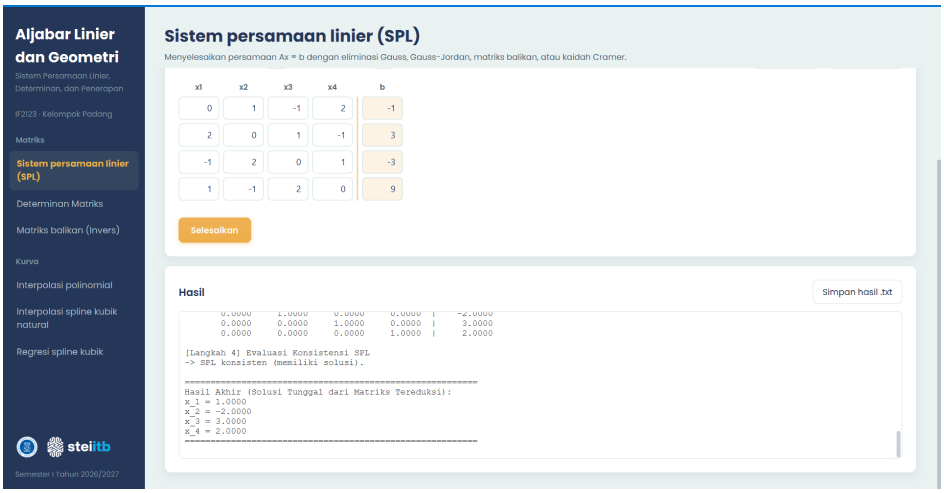
Matriks Masukan:

$$\left[\begin{array}{cccc|c} 0 & 1 & -1 & 2 & -1 \\ 2 & 0 & 1 & -1 & 3 \\ -1 & 2 & 0 & 1 & -3 \\ 1 & -1 & 2 & 0 & 9 \end{array}\right]$$

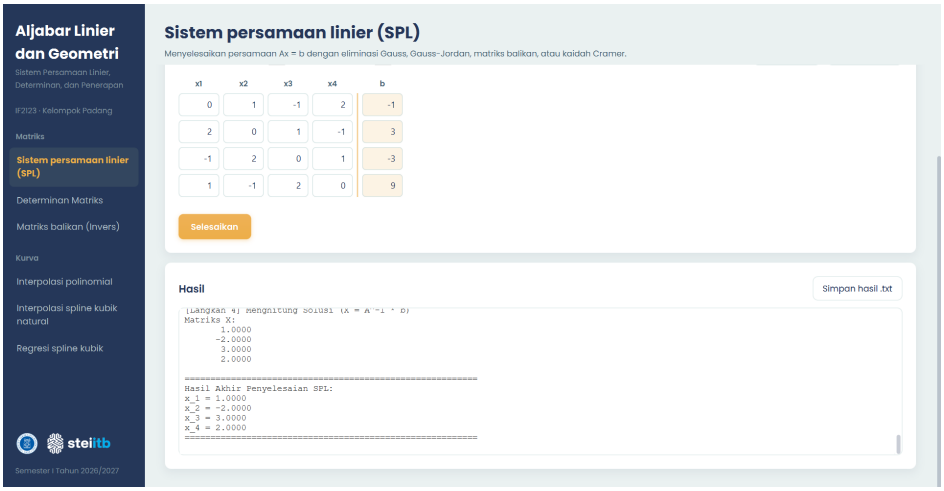
Hasil Eksekusi Program:



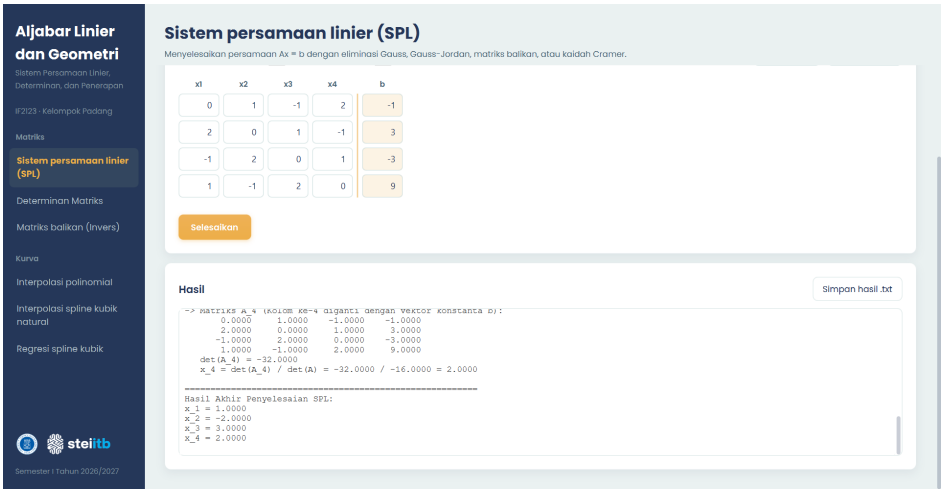
Gambar 4.31: Penyelesaian SPL dengan Metode Gauss



Gambar 4.32: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.33: Penyelesaian SPL dengan Metode Invers



Gambar 4.34: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan

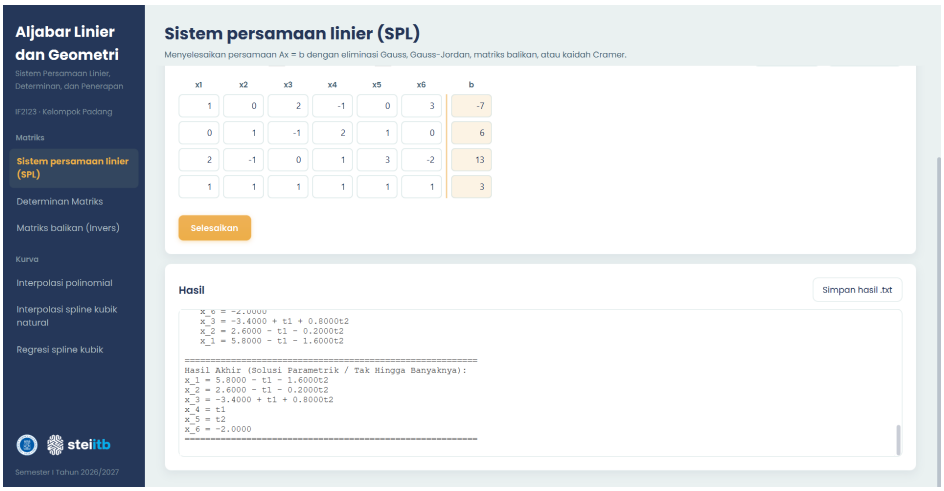
Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan matriks koefisien tidak bernilai nol, matriks memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

4.4.2 Kasus Uji 4.4.2: Matriks augmented A Berukuran 4×6

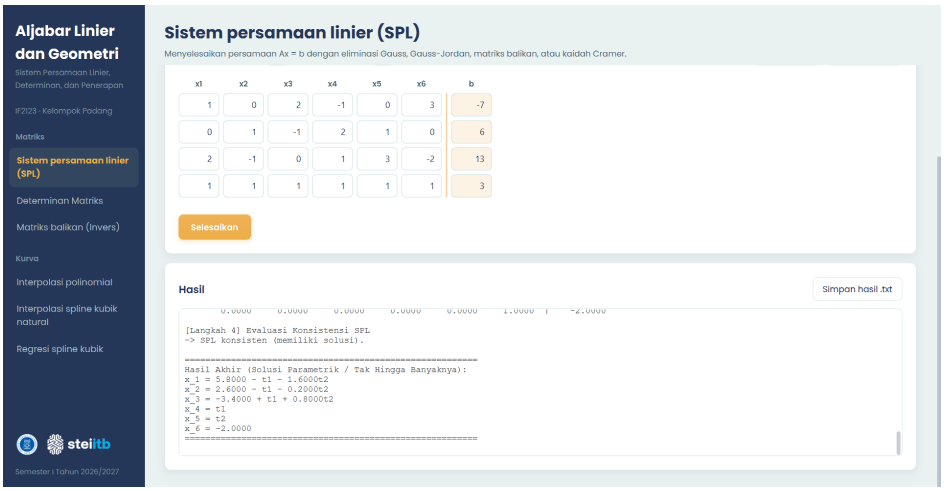
Matriks Masukan:

$$\left[\begin{array}{cccccc|c} 1 & 0 & 2 & -1 & 0 & 3 & -7 \\ 0 & 1 & -1 & 2 & 1 & 0 & 6 \\ 2 & -1 & 0 & 1 & 3 & -2 & 13 \\ 1 & 1 & 1 & 1 & 1 & 1 & 3 \end{array}\right]$$

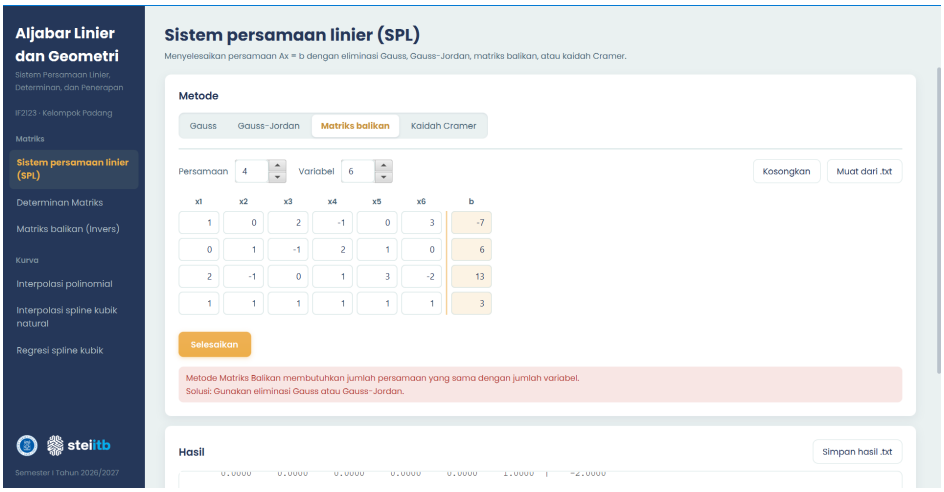
Hasil Eksekusi Program:



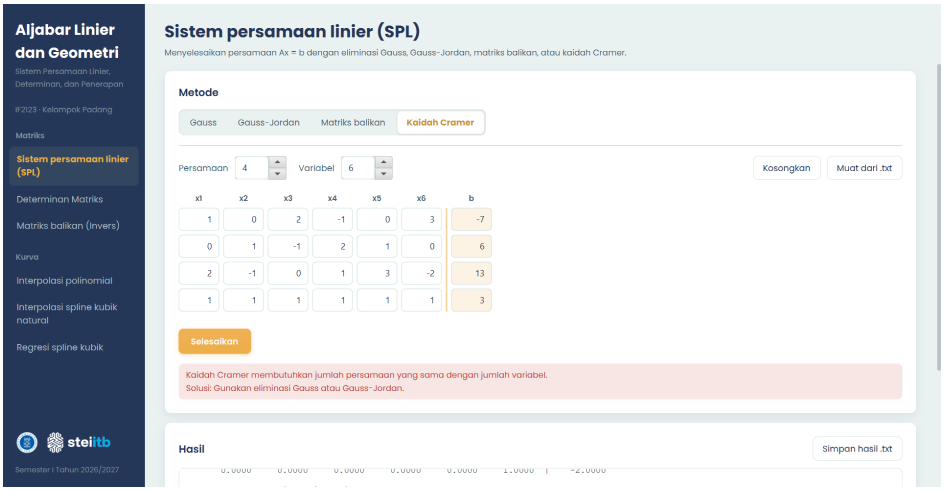
Gambar 4.35: Penyelesaian SPL dengan Metode Gauss



Gambar 4.36: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.37: Penyelesaian SPL dengan Metode Invers



Gambar 4.38: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Matriks memiliki jumlah variabel yang lebih banyak daripada persamaan tidak memenuhi syarat Kaidah Cramer dan Metode Matriks Balikan sehingga metode yang dapat digunakan hanya metode Eliminasi Gauss dan Gauss-Jordan.

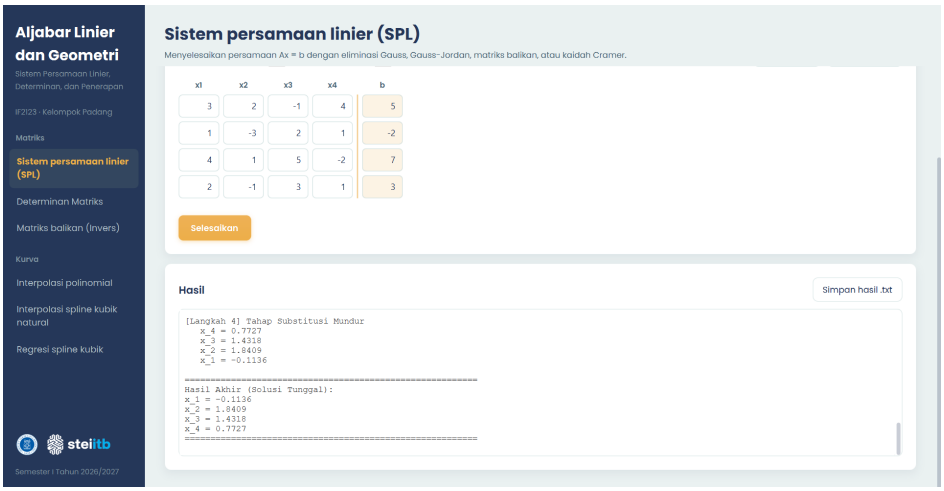
4.5 Kasus Uji 4.5: Sistem Persamaan Linier Berbentuk Umum

4.5.1 Kasus Uji 4.5.1: SPL 4×4

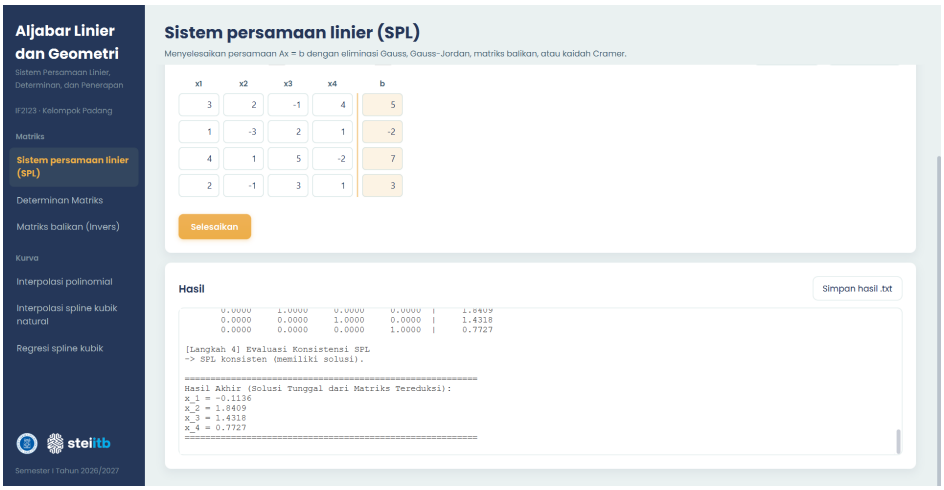
Sistem Persamaan Masukan:

$$\begin{aligned} 3x_1 + 2x_2 - x_3 + 4x_4 &= 5 \\ x_1 - 3x_2 + 2x_3 + x_4 &= -2 \\ 4x_1 + x_2 + 5x_3 - 2x_4 &= 7 \\ 2x_1 - x_2 + 3x_3 + x_4 &= 3 \end{aligned}$$

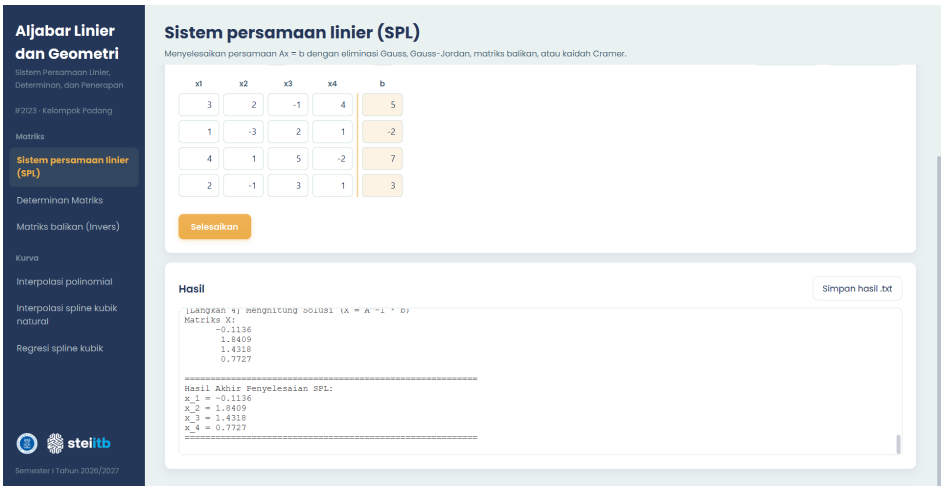
Hasil Eksekusi Program:



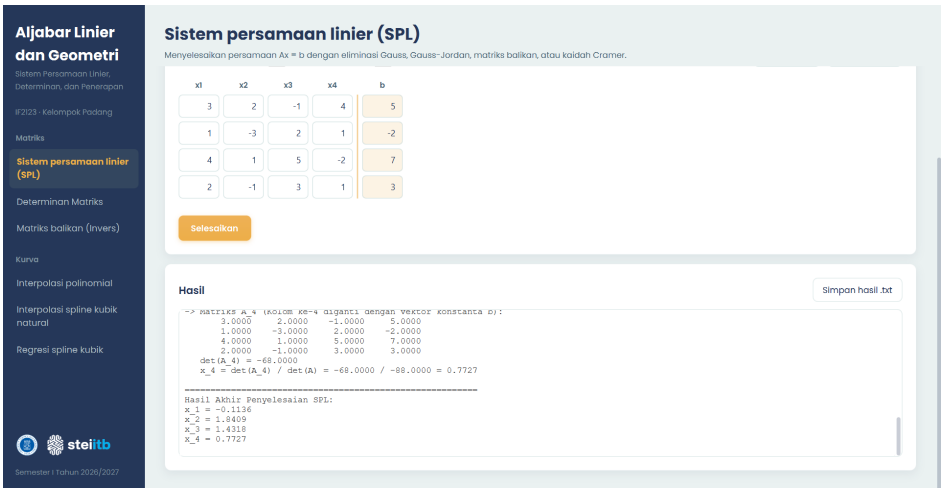
Gambar 4.39: Penyelesaian SPL dengan Metode Gauss



Gambar 4.40: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.41: Penyelesaian SPL dengan Metode Invers



Gambar 4.42: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan SPL tidak bernilai nol, SPL memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

4.5.2 Kasus Uji 4.5.2: SPL 9×9

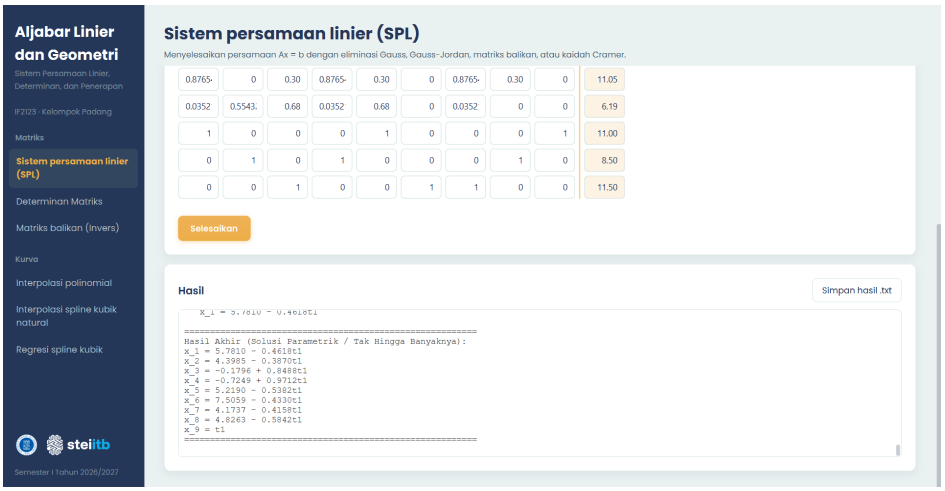
Sistem Persamaan Masukan (Bentuk Asli):

$$
\begin{aligned}
x_1 + x_2 + x_3 &= 10.00 \\
x_4 + x_5 + x_6 &= 12.00 \\
x_7 + x_8 + x_9 &= 9.00 \\
0.03521(x_1 + x_4 + x_7) + 0.68(x_2 + x_6) + 0.55432x_9 &= 8.42 \\
0.87654(x_1 + x_4 + x_7) + 0.30(x_3 + x_5 + x_8) &= 11.05 \\
0.03521(x_1 + x_4 + x_7) + 0.68(x_3 + x_5) + 0.55432x_2 &= 6.19 \\
x_1 + x_5 + x_9 &= 11.00 \\
x_2 + x_4 + x_8 &= 8.50 \\
x_3 + x_6 + x_7 &= 11.50
\end{aligned}
$$

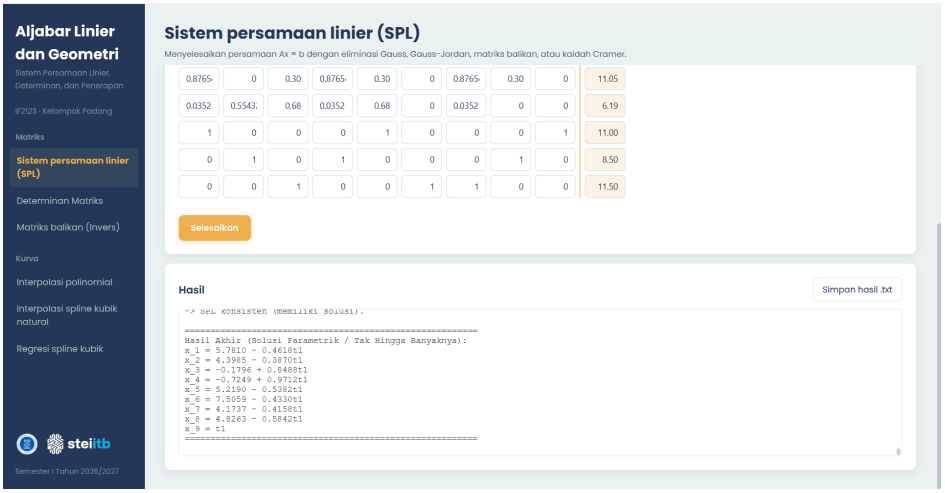
Penjabaran Persamaan:

$$
\begin{aligned}
1x_1 + 1x_2 + 1x_3 + 0x_4 + 0x_5 + 0x_6 + 0x_7 + 0x_8 + 0x_9 &= 10.00 \\
0x_1 + 0x_2 + 0x_3 + 1x_4 + 1x_5 + 1x_6 + 0x_7 + 0x_8 + 0x_9 &= 12.00 \\
0x_1 + 0x_2 + 0x_3 + 0x_4 + 0x_5 + 0x_6 + 1x_7 + 1x_8 + 1x_9 &= 9.00 \\
0.03521x_1 + 0.68x_2 + 0x_3 + 0.03521x_4 + 0x_5 + 0.68x_6 + 0.03521x_7 + 0x_8 + 0.55432x_9 &= 8.42 \\
0.87654x_1 + 0x_2 + 0.30x_3 + 0.87654x_4 + 0.30x_5 + 0x_6 + 0.87654x_7 + 0.30x_8 + 0x_9 &= 11.05 \\
0.03521x_1 + 0.55432x_2 + 0.68x_3 + 0.03521x_4 + 0.68x_5 + 0x_6 + 0.03521x_7 + 0x_8 + 0x_9 &= 6.19 \\
1x_1 + 0x_2 + 0x_3 + 0x_4 + 1x_5 + 0x_6 + 0x_7 + 0x_8 + 1x_9 &= 11.00 \\
0x_1 + 1x_2 + 0x_3 + 1x_4 + 0x_5 + 0x_6 + 0x_7 + 1x_8 + 0x_9 &= 8.50 \\
0x_1 + 0x_2 + 1x_3 + 0x_4 + 0x_5 + 1x_6 + 1x_7 + 0x_8 + 0x_9 &= 11.50
\end{aligned}
$$

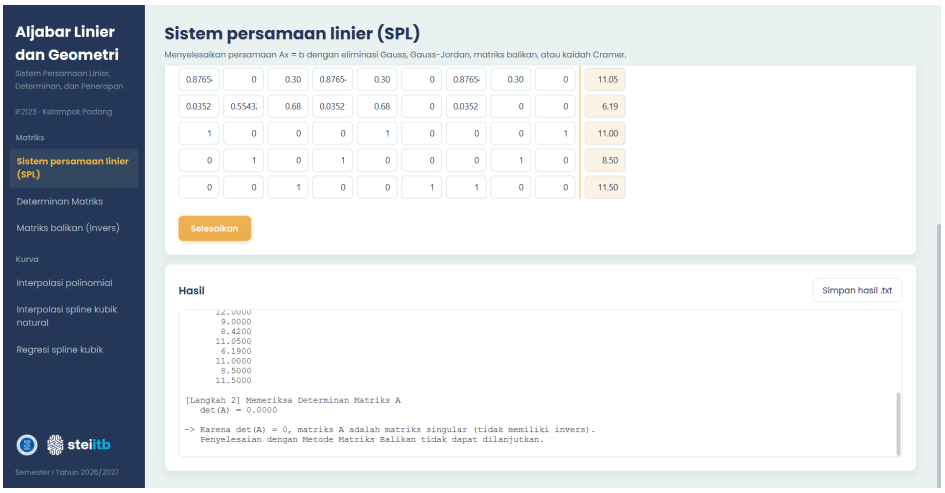
Hasil Eksekusi Program:



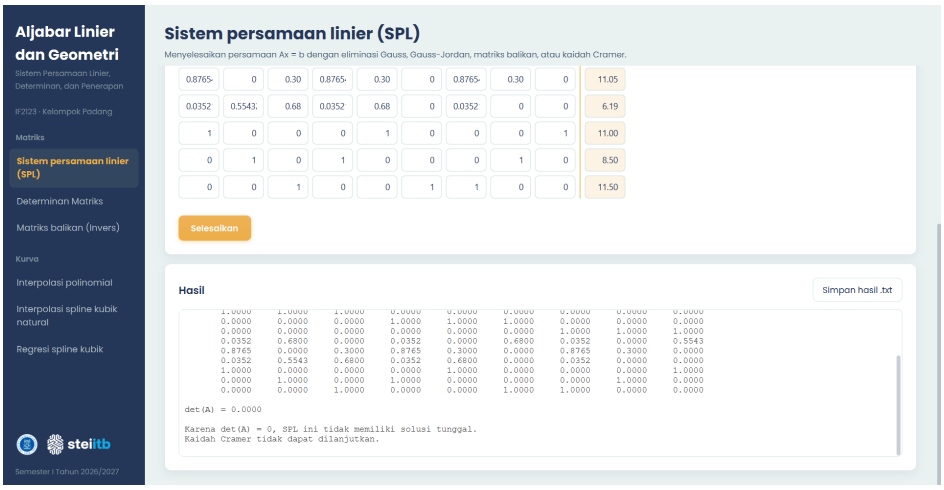
Gambar 4.43: Penyelesaian SPL dengan Metode Gauss



Gambar 4.44: Penyelesaian SPL dengan Metode Gauss-Jordan



Gambar 4.45: Penyelesaian SPL dengan Metode Invers



Gambar 4.46: Penyelesaian SPL dengan Kaidah Cramer

Analisis: Berdasarkan hasil perhitungan dari metode Eliminasi Gauss dan Gauss-Jordan, SPL A memiliki solusi parametrik/tak hingga, tetapiSPL memiliki $determinan = 0$ sehingga metode Kaidah Cramer dan Metode Matriks Balikan tidak dapat digunakan.

4.6 Kasus Uji 4.6: Aplikasi Sistem Persamaan Linier

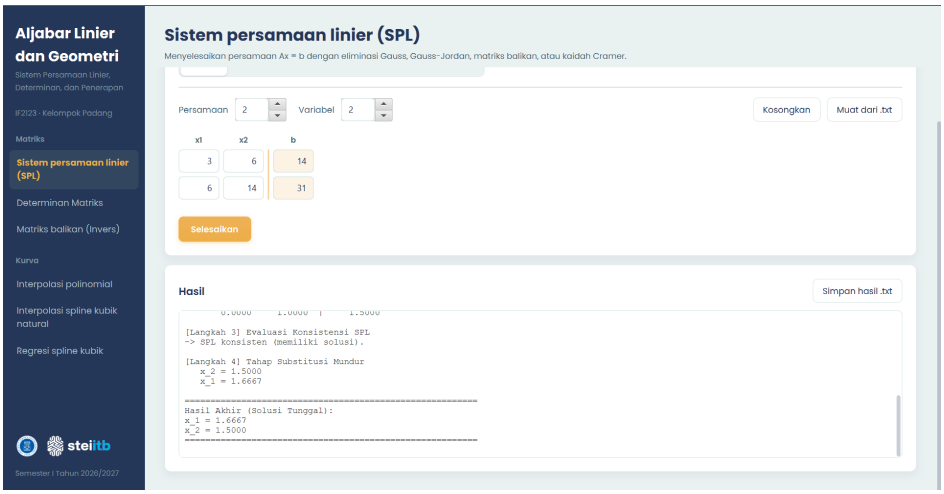
Persamaan Masukan (Hasil Penjabaran): Substitusikan nilai dari dataset ke dalam persamaan:

$$3\theta_0 + (1+2+3)\theta_1 = 3+5+6$$
$$(1+2+3)\theta_0 + (1^2+2^2+3^2)\theta_1 = (1)(3)+(2)(5)+(3)(6)$$

Sehingga sistem persamaan linear yang siap dimasukkan ke kalkulator (dengan memisalkan $x_1 = \theta_0$ dan $x_2 = \theta_1$) adalah:

$$3x_1 + 6x_2 = 14$$
$$6x_1 + 14x_2 = 31$$

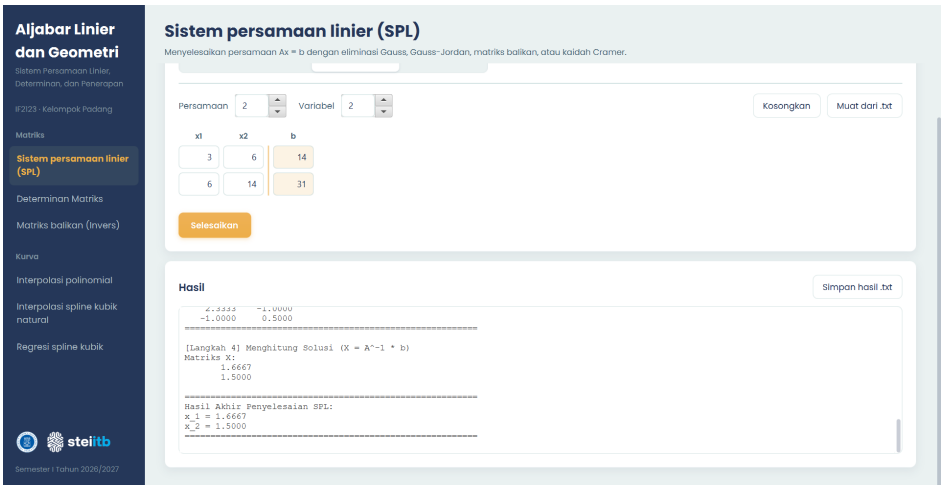
Hasil Eksekusi Program:



Gambar 4.47: Hasil Metode Gauss Kasus Uji 5.6



Gambar 4.48: Hasil Metode Gauss-Jordan Kasus Uji 5.6



Gambar 4.49: Hasil Metode Invers Kasus Uji 5.6



Gambar 4.50: Hasil Metode Cramer Kasus Uji 5.6

Analisis: Seluruh metode, yaitu Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan, menghasilkan solusi tunggal yang konsisten. Karena determinan SPL tidak bernilai nol, SPL memiliki invers sehingga Kaidah Cramer dan Metode Matriks Balikan dapat diterapkan secara valid.

4.7 Kasus Uji 4.7: Interpolasi Polinomial

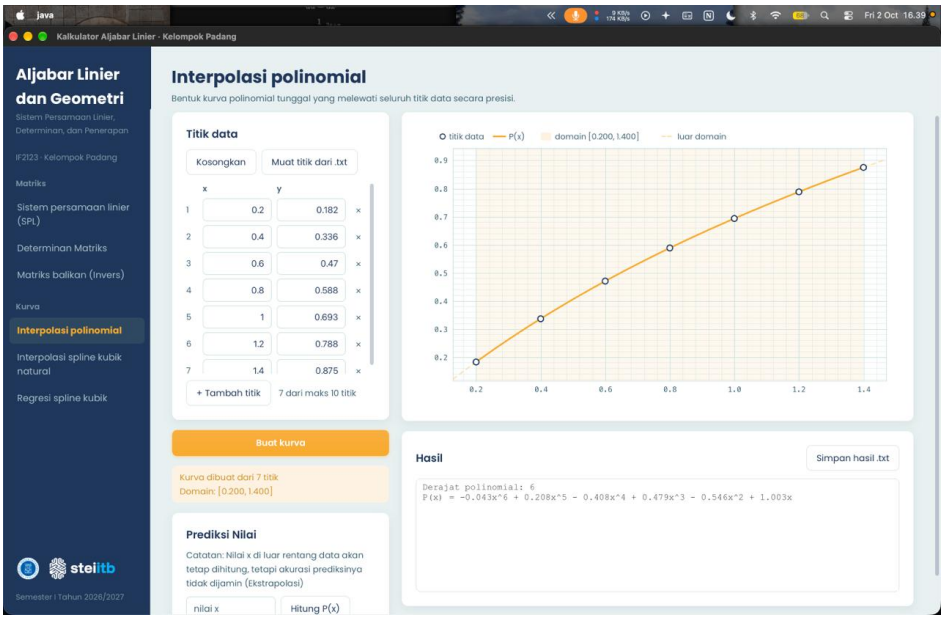
4.7.1 Kasus Uji 4.7.1: Tujuh Titik Data (Domain $[0{,}2;\ 1{,}4]$)

Titik Data Masukan:

x	0,2	0,4	0,6	0,8	1,0	1,2	1,4
y	0,182	0,336	0,47	0,588	0,693	0,788	0,875

Tabel 4.1: Titik Data Masukan Kasus Uji 4.7.1

Hasil Eksekusi Program:



Gambar 4.51: Hasil Interpolasi Polinomial Kasus Uji 5.7.1

Analisis: Program menerima tujuh titik data sehingga polinomial interpolasi yang dihasilkan berderajat enam, yaitu

$$P(x) = -0{,}043x^6 + 0{,}208x^5 - 0{,}408x^4 + 0{,}479x^3 - 0{,}546x^2 + 1{,}003x.$$

Kurva $P(x)$ melewati seluruh titik data pada domain $[0{,}2;\ 1{,}4]$ dan berbentuk menaik secara mulus dengan kelengkungan yang makin landai, menyerupai pola fungsi logaritmik. Bagian kurva di luar domain digambar putus-putus karena nilainya merupakan hasil ekstrapolasi, sehingga akurasinya tidak dijamin.

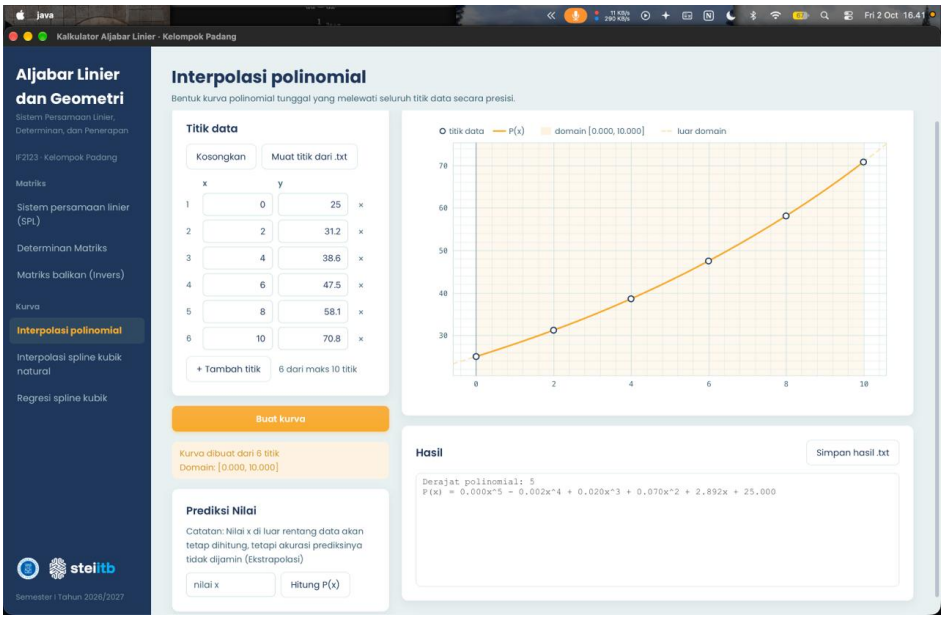
4.7.2 Kasus Uji 4.7.2: Enam Titik Data (Domain $[0;\ 10]$)

Titik Data Masukan:

x	0	2	4	6	8	10
y	25	31,2	38,6	47,5	58,1	70,8

Tabel 4.2: Titik Data Masukan Kasus Uji 4.7.2

Hasil Eksekusi Program:



Gambar 4.52: Hasil Interpolasi Polinomial Kasus Uji 4.7.2

Analisis: Enam titik data menghasilkan polinomial berderajat lima, yaitu

$$P(x) = 0{,}000x^5 - 0{,}002x^4 + 0{,}020x^3 + 0{,}070x^2 + 2{,}892x + 25{,}000.$$

Suku x^5 bernilai sangat kecil sehingga tampil sebagai $0{,}000$ pada pembulatan tiga angka desimal, dan suku konstan $25{,}000$ sesuai dengan titik pertama $(0,\ 25)$. Kurva melewati seluruh titik data dan menunjukkan tren naik yang makin curam seiring bertambahnya x.

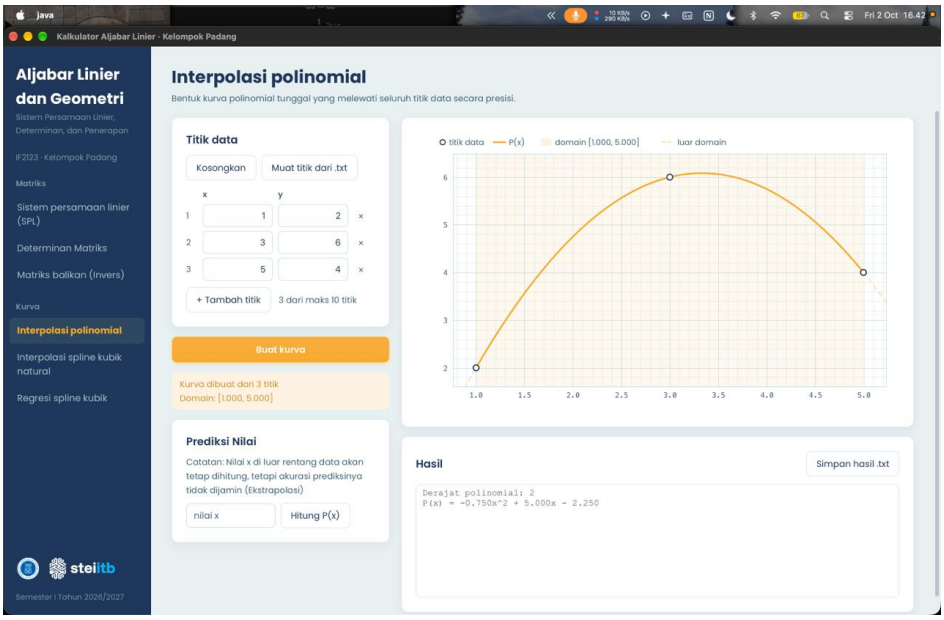
4.7.3 Kasus Uji 4.7.3: Tiga Titik Data

Titik Data Masukan:

x	1	3	5
y	2	6	4

Tabel 4.3: Titik Data Masukan Kasus Uji 4.7.3

Hasil Eksekusi Program:



Gambar 4.53: Hasil Interpolasi Polinomial Kasus Uji 4.7.3

Analisis: Tiga titik data menghasilkan polinomial berderajat dua, yaitu

$$P(x) = -0{,}750x^2 + 5{,}000x - 2{,}250.$$

Substitusi $x = 1$, $x = 3$, dan $x = 5$ menghasilkan $P(x) = 2$, 6, dan 4, sehingga kurva tepat melewati ketiga titik. Koefisien x^2 yang negatif membuat parabola membuka ke bawah dengan titik puncak di sekitar $x \approx 3{,}33$ dan $P(x) \approx 6{,}083$, yang sesuai dengan bentuk kurva pada grafik.

4.8 Kasus Uji 4.8: Studi Interpolasi Natural Cubic Spline

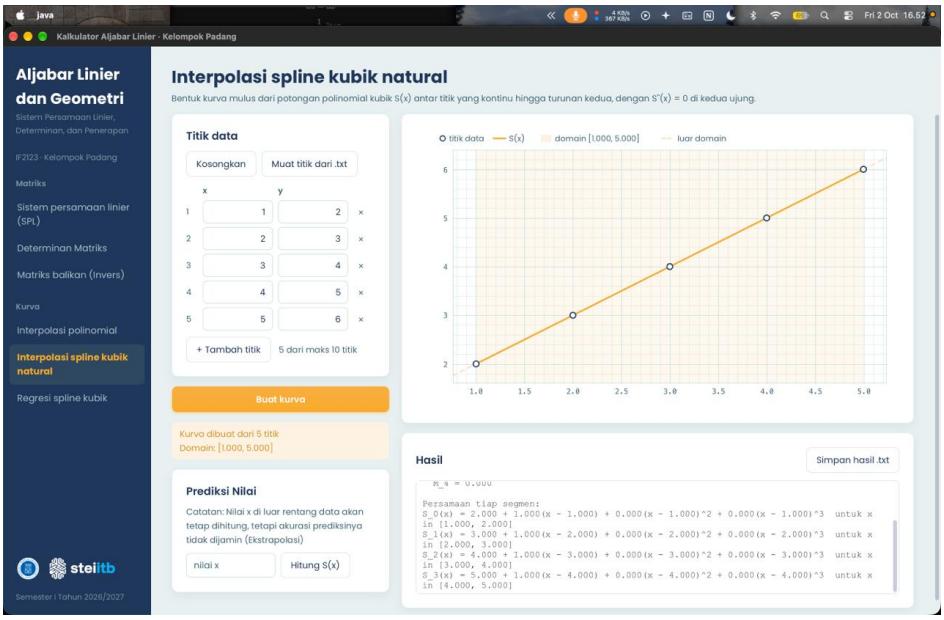
4.8.1 Kasus Uji 4.8.1: Lima Titik Data

Titik Data Masukan:

x	1	2	3	4	5
y	2	3	4	5	6

Tabel 4.4: Titik Data Masukan Kasus Uji 4.8.1

Hasil Eksekusi Program:



Gambar 4.54: Hasil Interpolasi Natural Cubic Spline Kasus Uji 4.8.1

Analisis: Lima titik data membentuk empat segmen polinomial kubik $S_0(x)$ sampai $S_3(x)$ pada selang $[1, 2]$, $[2, 3]$, $[3, 4]$, dan $[4, 5]$. Setiap segmen berbentuk

$$S_j(x) = a_j + b_j(x - x_j) + c_j(x - x_j)^2 + d_j(x - x_j)^3,$$

dengan $b_j = 1{,}000$ serta $c_j = d_j = 0{,}000$ pada seluruh segmen, misalnya $S_0(x) = 2 + 1(x - 1)$. Titik-titik data ini terletak pada garis lurus $y = x + 1$, sehingga seluruh nilai momen kedua M_i yang memenuhi syarat *natural* ($M_0 = M_n = 0$) bernilai nol dan spline yang diperoleh berimpit dengan garis tersebut. Hasil ini sesuai dengan kurva pada grafik, yaitu garis lurus yang melewati seluruh titik data.

4.9 Kasus Uji 4.9: Studi Kasus Regresi Spline

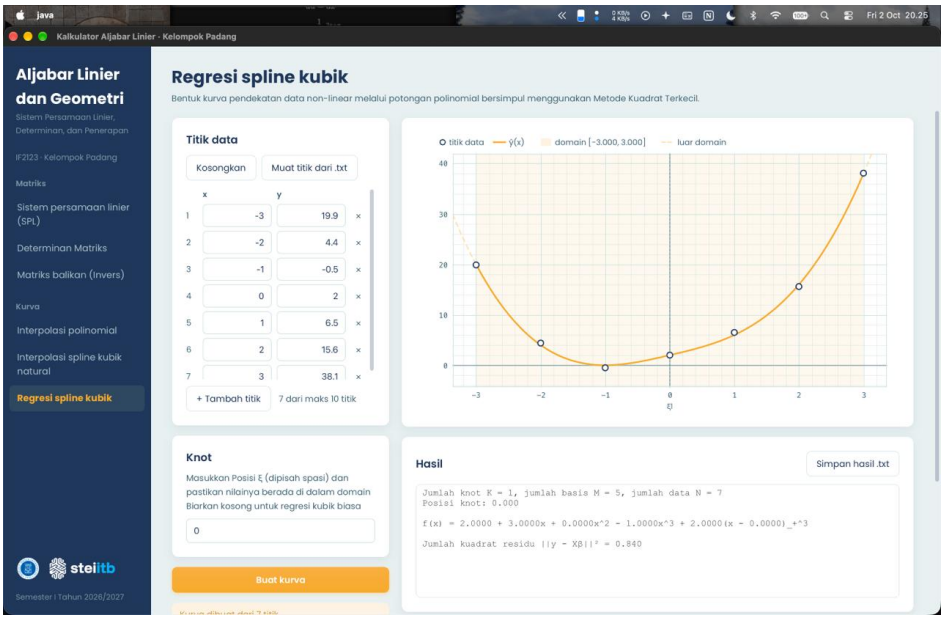
4.9.1 Kasus Uji 4.9.1

Dataset Masukan: Dataset terdiri dari $N = 7$ titik data berjarak seragam pada domain $[-3, 3]$ dengan pola non-linear (menurun lalu naik tajam). Regresi spline kubik dilakukan dengan satu knot di posisi $\xi = 0$ ($K = 1$), sehingga jumlah basis $M = K + 4 = 5$.

i	x	y
1	-3	19,9
2	-2	4,4
3	-1	$-0{,}5$
4	0	2,0
5	1	6,5
6	2	15,6
7	3	38,1

Tabel 4.5: Dataset Masukan Studi Kasus Regresi Spline

Hasil Eksekusi Program:



Gambar 4.55: Hasil Regresi Spline Studi Kasus 4.9.1

Program menghasilkan fungsi regresi

$$f(x) = 2 + 3x + 0x^2 - x^3 + 2\,(x-0)_+^3,$$

dengan jumlah kuadrat residual $\|\mathbf{y} - X\boldsymbol{\beta}\|^2 = 0{,}840$. Dalam bentuk piecewise:

$$f(x) = \begin{cases} 2 + 3x - x^3, & x < 0, \\ 2 + 3x + x^3, & x \geq 0. \end{cases}$$

x	y	$f(x)$	**Residual**
-3	19,9	20,0	$-0{,}1$
-2	4,4	4,0	0,4
-1	$-0{,}5$	0,0	$-0{,}5$
0	2,0	2,0	0,0
1	6,5	6,0	0,5
2	15,6	16,0	$-0{,}4$
3	38,1	38,0	0,1

Tabel 4.6: Perbandingan Data Aktual dan Hasil Regresi Spline

Analisis: Kurva hasil regresi mengikuti pola data dengan sangat baik. Kurva turun dari $x = -3$ menuju titik minimum lokal di sekitar $x = -1$ (dengan $f(-1) = 0$), kemudian naik semakin curam hingga $x = 3$. Sisi kanan knot lebih curam karena suku $2(x-\xi)_+^3$ menambah kontribusi kubik hanya untuk $x \geq \xi$, sehingga koefisien x^3 berubah dari -1 (kiri knot) menjadi $+1$ (kanan knot).

Kesesuaian terhadap data ditunjukkan oleh residual yang kecil, dengan nilai mutlak maksimum $0{,}5$ dan jumlah kuadrat residual hanya $0{,}840$ untuk 7 titik data. Koefisien x^2 bernilai

0, dan kurva tetap kontinu serta mulus (C^2) di knot $\xi = 0$ karena menggunakan basis truncated power. Hal ini menunjukkan bahwa satu knot sudah cukup untuk menangkap perubahan kelengkungan data, dan penambahan knot kemungkinan hanya berisiko overfitting. Bagian kurva putus-putus di luar domain $[-3, 3]$ merupakan ekstrapolasi dan kurang dapat diandalkan karena tidak didukung data.

5 Penutup

5.1 Kesimpulan

Tugas Besar Aljabar Linear dan Geometri telah berhasil mengimplementasikan berbagai konsep aljabar linear ke dalam sistem berbasis Java, meliputi perhitungan determinan, invers matriks, serta penyelesaian Sistem Persamaan Linear (SPL) dengan beberapa metode. Melalui tugas besar ini, diperoleh pemahaman mengenai penerapan konsep matematika ke dalam algoritma dan pemrograman berorientasi objek, sekaligus pengalaman dalam menyusun dokumentasi teknis menggunakan LaTeX.

5.2 Saran

Pengembangan selanjutnya dapat difokuskan pada peningkatan interaktivitas pada bagian output sistem. Saat ini, hasil perhitungan masih ditampilkan secara statis sehingga pengguna hanya dapat melihat hasil akhir dari proses. Sistem dapat dikembangkan dengan menampilkan ilustrasi atau tahapan perhitungan secara bertahap, sehingga pengguna dapat memahami proses penyelesaian, bukan hanya memperoleh hasil akhirnya. Pengembangan ini diharapkan dapat meningkatkan aspek interaktif dan membantu pengguna dalam memahami penerapan setiap metode aljabar linear yang digunakan.

5.3 Refleksi Anggota Kelompok

Ghina Emelia Yantes (13525119)

Selama mengerjakan Tugas Besar Aljabar Linear dan Geometri ini, saya menjadi makin paham dengan materi di kelas. Saya juga mempelajari materi baru pada penerapan interpolasi dan regresi kubik. Selain itu, saya juga belajar cara membuat GUI menggunakan Javafx yang menurut saya sangat menarik karena membuat saya mencoba lingkungan bahasa baru, yaitu Java. Hal lain yang yang saya dapatkan adalah bagaimana bekerja dengan tim dalam membuat sebuah project, bagaimana membagi tugas, koordinasi, dan sebagainya.

Sahla Nailah Salsabilla (13525134)

Selama mengerjakan Tugas Besar Aljabar Linear dan Geometri ini, saya berfokus pada pengerjaan kode untuk susunan matriks dan operasi dasarnya, beserta invers dan determinan. Proses ini menuntut saya untuk memahami konsep Object-Oriented Programming (OOP) dalam Java agar kode yang saya buat untuk matriks dapat digunakan kembali (reusable) untuk berbagai ukuran matriks. Saya juga belajar format penulisan dalam LaTeX guna merancang struktur laporan yang rapi agar hasil kerja saya dapat dipahami dengan baik oleh pembaca.

Nayla Putri Ghaisani (13525140)

Selama mengerjakan Tugas Besar Aljabar Linear dan Geometri ini, saya memperoleh berbagai pengalaman dan pengetahuan baru, khususnya dalam penerapan konsep **Object-Oriented Programming (OOP)** menggunakan bahasa pemrograman Java melalui penggunaan class dan objek. Selain itu, saya juga mempelajari penggunaan **LaTeX** untuk menyusun dan memformat laporan secara terstruktur, sistematis, dan rapi. Pengalaman ini tidak hanya membantu saya memahami materi yang dipelajari, tetapi juga meningkatkan kemampuan saya dalam mengimplementasikan konsep pemrograman serta menyusun dokumentasi teknis dengan baik.

6 Daftar Pustaka

[1] H. Anton and C. Rorres, *Elementary Linear Algebra: Applications Version*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2010. [Online]. Available: https://archive.org/details/ElementaryLinearAlgebraByHowardAnton10thEdition

[2] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, Cengage Learning, 2011, ch. 3 (Interpolation and Polynomial Approximation). [Online]. Available: https://faculty.ksu.edu.sa/sites/default/files/numerical_analysis_9th.pdf

[3] G. Muntingh, "Topics in Polynomial Interpolation Theory," Ph.D. dissertation, Centre of Mathematics for Applications, University of Oslo, Oslo, Norway, Dec. 2010. [Online]. Available: https://www.researchgate.net/publication/265819309_Topics_in_Polynomial_Interpolation_Theory

[4] R. L. Burden and J. D. Faires, *Numerical Analysis*, 9th ed. Boston, MA, USA: Brooks/Cole, Cengage Learning, 2011, ch. 3, sec. 3.5 (Cubic Splines). [Online]. Available: https://faculty.ksu.edu.sa/sites/default/files/numerical_analysis_9th.pdf

[5] C. de Boor, *A Practical Guide to Splines*, rev. ed. New York, NY, USA: Springer-Verlag, 2001. [Online]. Available: https://doi.org/10.1007/978-1-4612-6333-3

[6] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. New York, NY, USA: Springer, 2009, ch. 5 (Basis Expansions and Regularization). [Online]. Available: https://course.ccs.neu.edu/ds4420sp20/readings/ESLII.pdf

[7] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: with Applications in R*, 2nd ed. New York, NY, USA: Springer, 2021, ch. 7 (Moving Beyond Linearity). [Online]. Available: https://www.statlearning.com/

[8] M. Bertalmio, G. Sapiro, V. Caselles, and C. Ballester, "Image Inpainting," in *Proceedings of the 27th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '00)*, New York, NY, USA: ACM, 2000, pp. 417–424. [Online]. Available: https://doi.org/10.1145/344779.344972

[9] P. Pérez, M. Gangnet, and A. Blake, "Poisson image editing," *ACM Transactions on Graphics (TOG)*, vol. 22, no. 3, pp. 313–318, 2003. [Online]. Available: https://doi.org/10.1145/882262.882269

7 Lampiran

7.1 Repositori GitHub

https://github.com/ghinayantes/algeo26-tb1-Padang.git

7.2 Bonus LaTeX

https://github.com/ghinayantes/algeo26-tb1-Padang/tree/main/docs/src

7.3 Pernyataan Integritas Akademik

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Ghina Emelia Yantes



Sahla Nailah Salsabilla



Nayla Putri Ghaisani